

离散数学 2

目录

离散数学 2	1
一、图论基础概念 2026-02-25	3
(1) 特别基础：图的顶点、边、方向	3
(2) 握手定理	4
(3) 赋权图	4
(4) 特殊的图	4
完全图：所有点之间互相连接	4
无向完全图	4
有向完全图	4
k -正则图	4
圈图	4
轮圈	4
二分图	4
Ramsey 问题简单记一下	4
(5) 子图	5
(6) 同构	5
同构的判断充要条件	5
(7) 图的运算	5
二、图的代数结构 2026-03-06	5
(1) 邻接矩阵	5
转置	5
乘积	6
(2) 关联矩阵	6
对于无向图，	6
对于有向图，	6
(3) 边列表	6
(4) 正向表	6
(5) 逆向表	7
(6) 邻接表	7
(7) 有向图的十字链表	7
三、道路与回路 2026-03-13	7
(1) 基本概念	7
(2) 图的连通性	8
(3) 割点、割边与连通度	9
(4) 块 (Block)	9
(5) 复杂网络简介	10
(6) 道路与回路的判定方法	10
(7) 搜索算法	10
四、特殊道路与回路问题 2026-03-17	11
(1) 欧拉道路与回路	11
基本概念与定义	11
欧拉图判别定理	12
构造欧拉回路的算法	12
欧拉道路判别定理	12
k 笔画问题	13
有向图的欧拉道路与回路	13
应用：计算机编码盘设计	13
(2) 哈密顿道路与回路	13
基本概念	13
哈密顿图与欧拉图的关系	14

哈密顿回路判定——基本性质	14
哈密顿回路判定——必要条件	14
哈密顿道路判定——充分条件	15
闭合图与判定定理	16
哈密顿图判定方法总结	16
(3) 欧拉回路与哈密顿回路的比较	16
(4) 判定定理汇总	16
欧拉回(道)路判定的充要条件	16
哈密顿回(道)路判定的必要条件	17
哈密顿回(道)路判定的充分条件	17
哈密顿回路充要条件	17
(5) 旅行商问题与分支定界法	17
“便宜”算法（近似解法）	17
(6) 最短路径	18
基本定义	18
重要引理	18
Dijkstra 算法	18
其他最短路径算法	19
最短路径算法总结	19
(7) 关键路径	20
引理与定理 2.7.1	20
拓扑排序	20
拓扑排序的实现	21
关键路径基本概念	23
PERT 图简介（选学）	23
最早启动时间	23
最晚启动时间与最大允许延误时间	23
(8) 中国邮路	24
问题提出	24
无向图的中国邮路	24
定理 2.8.1	24
奇偶点图上作业法	24
Edmonds 最小权匹配算法	25
有向图的中国邮路	25
💧 图论：血泪教训与阴间套路总结 💧	25
1. 核心铁律：只要边数够多，必有妖孽	25
2. 情况 B 的必杀技：最长路径论证 (Longest Path Argument)	25
3. 无三角形 (Triangle-free) 的物理限制	25
4. 考试/自救心态建设	26
五、树 2026-04-09	26
(1) 树的定义	26
(2) 树的等价定义	26
证明	26
(3) 支撑树	27
生成支撑树的方法	27
生成所有支撑树的方法	27
最小生成树	28
(4) 基本关联矩阵及其性质	28
线性相关预备知识	28
关联矩阵	28
关联矩阵的秩	29
基本关联矩阵	29
(5) 支撑树的计数	30

Binet-Cauchy 定理	30
支撑树计数公式	30
特殊情况计数	31
(6) 根树	31
根树计数	32
(7) 回路矩阵	32
完全回路矩阵	32
基本回路矩阵	33
回路矩阵的性质	33
(8) 割集矩阵	35
边割集	35
完全割集矩阵	35
基本割集矩阵	36
割集矩阵的性质	36
矩阵关系总结 (必考!)	37
(9) 最短树	37
最小支撑树	37
Prim 算法	37
Kruskal 算法	38
(10) Huffman 树	38
二叉树	38
最优二叉树	38
Huffman 算法	39
Huffman 编码	40
六、平面图与对偶图 2026-04-23	40
(1) 平面图	40
平面图定义 (定义 4.1.1)	40
面 (定义 4.1.2)	40
面的次数定理	41
(2) 极大平面图	41
定义与性质	41
(3) 图的平面性检测	41
非平面图	41
同胚与边收缩	41
Kuratowski 定理	41
(4) 对偶图	42
对偶图的定义	42
对偶图的性质	42
自对偶图	42

一、图论基础概念 2026-02-25

(1) 特别基础: 图的顶点、边、方向

图 G 主要由两部分组成:

1. 顶点集合 V , 里面的东西是节点或者顶点
2. 边集合 E , 是 V 上的二元关系, 顶点集合的笛卡尔积上的多重集 (即可以有重复的元素)
3. 表示方法

$$V = \{v_1, v_2, v_3, \dots, v_n\}$$

$$E = \{(v_1, v_2), \dots, (v_4, v_5)\}$$

4. 对有限图 $G = (V, E)$

1. 阶 = $|V| = n$
2. $|E| = 0$ 空图
3. $|V| = 1$ 平凡图

4. 未说明是默认有限无向图 一些举例：食物链是有向图

关联同时存在是就是无向图，否则可能是有向图。

5. 无向图不等价于两边都有的有向图

5. 对无向图： 1.认为顶点作为端点时，和它的边有**关联**；普通边关联次数为 1，自环关联次数为 2，不相连关联次数为 0

2. 环、孤立点

3. **顶点相邻**：两个顶点有公共边，和一个点相邻的所有点组成它的邻点集

4. 边相邻：两条边有公共顶点 5. 重边 6. 几种分类：简单图：无重边无自环

多重图：有重边有自环 伪图：都有 7. 度数：所连边的数量，自环贡献两个度；或者就叫**与其相邻边的关联次数之和**

6. 有向图：

1. 通常用 $\langle v_1, v_2 \rangle$

2. 起点/直接前驱**邻接到**终点/直接后继，反之叫做**邻接于**

3. 直接后继集也是**外邻集**

4. 直接前驱集也是**内邻集**

5. 度数：上面**加粗文**；有入度和出度

7. Δ, δ ：最大度最小度 8. 度数列（同样的有入度列、出度列）：

(2) 握手定理

1. **任意图的所有顶点度数之和是边数的 2 倍，有向图的入度=出度=边数**

2. **!!! 任意图的奇度顶点的个数一定是偶数**

有神秘证明，简单理解：度数之和必然为偶数，偶度顶点度数之和必然为偶数，所以奇度顶点度数之和必然为偶数，所以一定有偶数个。

eg.（抽象为图结构的一个例子）不存在具有奇数个面且每个面都具有奇数条棱的多面体。 **面->点；棱->边**

3. **一个非空简单图，一定存在度数相同的顶点**

数学归纳法：不存在孤立点、存在一个、存在两个。显然存在两个已经解决了。不存在和存在一个不赖，因为每个点的度数取值范围总比此时的顶点数少 1，所以肯定有相同的。

(3) 赋权图

$w(e)$ 就是 e 的边权。

(4) 特殊的图

完全图：所有点之间互相连接

无向完全图

$$|V_{K_n}| = n(n-1)/2$$

有向完全图

$$m = n(n-1), \Delta^+ = \delta^+ = \delta^- = \Delta^- = n-1, \Delta = \delta = 2(n-1)$$

k -正则图

每个顶点度数均为 k ，边数为 $m = kn/2$

圈图

圈图至少 3 个顶点，并且是 2-正则图。

轮圈

在圈图里面放一个点，和圈上的所有点相连。

二分图

两部分只要非空就可以。完全二分图： $K_{m,n}$ **二分图可能有多种剖分方式。**

Ramsey 问题简单记一下

本质解决方法就是画出来图，然后分析实线/虚线多边形的存在可能性。

(5) 子图

1. 定义: $V' \subseteq V, E' \subseteq E$, 此时 $G' \subseteq G$
2. 真子图: $V' \subset V, E' \subset E$
3. 支撑 / 生成子图: 在定义基础上, $V' = V$
4. 导出子图: 包含某个点集内部所有**相连的边集** / 某个边集关联的**所有顶点集**。记作 $G[V'], G[E']$ 。

(6) 同构

存在双射函数, 将节点重新编号, 一一对应。 $G_1 \cong G_2$

判断同构的一些必要条件:

- 扭曲变形后和原图一致
- 顶点数、边数相同
- 度数列无序相同
- 对应顶点关联集和邻域相同
- 存在同构的导出子图
- **内部存在一样的封闭图形**

eg. 4 阶 3 边无向简单图。

根据握手定理, 总度数为 6, 故最大顶点度数为 3, 奇度点偶数个。现在我们列出来度数列就行了, 然后大概枚举一下。

eg. K_4 的生成子图按照边数划分, 有 1 1 2 3 2 1 1。这对称是因为——补图同构!

同构的判断充要条件

- 对于简单图, 当且仅当补图同构
- 当且仅当邻接矩阵可以通过交换行列得到
- 所有子图都**同构**

(7) 图的运算

1. 交: $G_1 \cap G_2 = (V_1 \cap V_2, E_1 \cap E_2)$
2. 并: $G_1 \cup G_2 = (V_1 \cup V_2, E_1 \cup E_2)$
3. 差: $G_1 - G_2 = (V_1, E_1 - E_2)$
4. 补: $\overline{G} = (V, E(K_n) - E)$
5. 笛卡尔积: $G_1 \times G_2 = (V_1 \times V_2, E)$, 其中 E 中的边满足: $(u_1, v_1), (u_2, v_2) \in E$ 当且仅当 $u_1 = u_2$ 且 $(v_1, v_2) \in E_2$ 或者 $v_1 = v_2$ 且 $(u_1, u_2) \in E_1$ 。
6. 自补图: G 与 $\overline{G}$ 同构
7. 对称差: $G_1 \oplus G_2 = (V_1 \cup V_2, E_1 \oplus E_2)$
8. $G - v$: 删除顶点 v 和与之关联的边
9. $G - e$: 删除边 e
10. $G + e$: 添加边 e , 如果 e 已经存在则不变
11. $G + v$: 添加顶点 v , 如果 v 已经存在则不变

二、图的代数结构 2026-03-06

(1) 邻接矩阵

对于有向图, 每行表示一个顶点的出边, 每列表示一个顶点的入边。对于无向图, 邻接矩阵是对称的。

对于简单图, 邻接矩阵中的元素是 0 或 1, 表示是否存在边。对于多重图, 元素可以是非负整数, 表示边的数量。

对于赋权图, 元素可以是边权。此时称为权矩阵, 为 0 时不连通。

转置

对于有向图, 邻接矩阵的转置对应于将图中的所有边的方向反转。

$$G' = (V, E'), E' = \{(v, w) | (w, v) \in E\}$$

乘积

邻接矩阵的乘积可以用来计算路径数量。例如， A^k 的元素 a_{ij}^k 表示从顶点 i 到顶点 j 的长度为 k 的路径数量。

(2) 关联矩阵

关联矩阵 M 是一个 n 行 m 列的矩阵，其中 n 是顶点数， m 是边数。

对于无向图，

如果边 e_j 连接顶点 v_i ，则 $M_{ij} = 1$ ；否则为 0。2 表示自环。

性质：

1. 从列来看， $\sum_{i=1}^n b_{ij} = 2, j = 1, 2, \dots, m$
2. 从行来看， $\sum_{j=1}^m b_{ij} = d(v_i), i = 1, 2, \dots, n$
3. 握手定理： $\sum_{i,j} b_{ij} = 2m$
4. 某列某元素为 2： e_j 是自环。
5. 相同的两列： e_j 和 e_k 是重边。
6. 某行全 0： v_i 是孤立点。

对于有向图，

如果边 e_j 从顶点 v_i 出发，则 $M_{ij} = 1$ ；如果边 e_j 到达顶点 v_i ，则 $M_{ij} = -1$ ；否则为 0。

设无环有向图 $D = \langle V, E \rangle$ ，令 $B(D) = (b_{ij})_{n \times m}$ ，满足

$$b_{ij} = \begin{cases} 1, & v_i \text{ 是边 } e_j \text{ 的起点} \\ 0, & v_i \text{ 与边 } e_j \text{ 无关} \\ -1, & v_i \text{ 是边 } e_j \text{ 的终点} \end{cases}$$

矩阵的问题：效率极低，无法利用稀疏矩阵的性质，无法直接反映图的结构特征。

怎么优化？

(3) 边列表

边列表有 2 个 m 维向量 A, B 组成，若 $e_k = (v_i, v_j)$ ，则 $A(k) = i, B(k) = j$ 。

举例：对于图 $G = (V, E)$ ，其中 $V = \{v_1, v_2, v_3\}$ ， $E = \{(v_1, v_2), (v_2, v_3)\}$ ，则边列表为 $A = [1, 2]$ ， $B = [2, 3]$ 。

特点：

- 适合存储稀疏图，节省空间。 $\mathcal{O}(nm) \Rightarrow \mathcal{O}(m)$
- 不适合存储密集图，查询效率低。枚举前驱后继效率低。

(4) 正向表

将节点从 0 到 $n-1$ 编号，将边列表按照起始节点排序，每一个节点的直接后继对应 B 中连续的一段下标区间。 A 的第 $n+1$ 项存储为 m ，表示边的总数。

正向表 (Forward Star) 结构图解

顶点 v	head[v]
1	1
2	3
3	4
4	6
5	7 (哨兵)

下标 i	终点 w	权重
1	v_2	5
2	v_3	2
3	v_4	8
4	v_1	1
5	v_5	3
6	—	—

区间对应 (指针指向)：

- 顶点 1 的边区间：edges[1] 到 edges[3-1]
- 顶点 2 的边区间：edges[3] 到 edges[4-1]
- 顶点 3 的边区间：edges[4] 到 edges[6-1]

特点:

1. $d^+(v_i) = A(i+1) - A(i)$
2. $A(i) = \sum_{j=0}^{i-1} d^+(v_j)$
3. 占用空间变成了 $\mathcal{O}(n+m)$, 适合存储稀疏图。
4. 查询效率高, 枚举直接后继效率高。
5. 前驱查询效率低。

(5) 逆向表

与正向表类似, 但按照边的终点排序, 每一个节点的直接前驱对应 B 中连续的一段下标区间。

无向图的逆向表和正向表是相同的。原因: 无向图中边没有方向, 正向表和逆向表的定义相同。

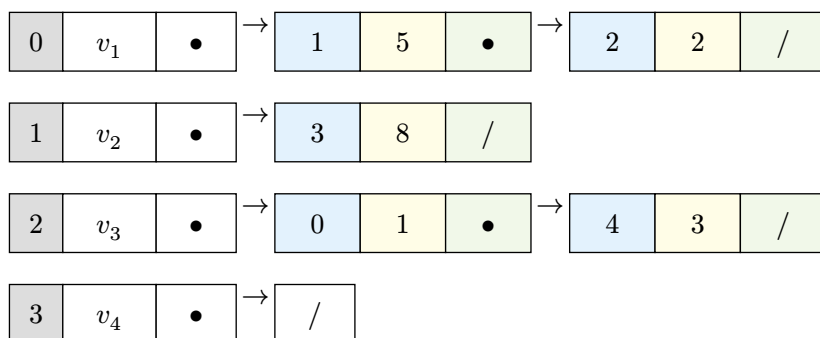
(6) 邻接表

每个节点维护一个链表, 链表中存储与该节点相邻的边的信息(如终点和权重)。对于无向图, 每条边会在两个节点的链表中出现; 对于有向图, 每条边只会在起点的链表中出现。起始节点只存储起始顶点。

- dst: 邻接点域
- w: 数据域
- link: 链域

邻接表 (Adjacency List) 存储结构

注: dst-邻接点域 | w-数据域 | link-链域



表头节点结构

data	firstedge
------	-----------

边节点结构

dst	w	link
-----	---	------

典型用途: 查询某个节点的直接后继或前驱。复杂度为 $O(d^+(v))$ 或 $O(d^-(v))$ 。

(7) 有向图的十字链表

节点结构如下:

- src: 边的起点
- dst: 边的终点
- weight: 边权
- d_link: 指向同一起点的下一条边
- s_link: 指向同一终点的下一条边

指针非常复杂。

三、道路与回路 2026-03-13

(1) 基本概念

1. **通路 (Path/Walk):** 给定图 $G = \langle V, E \rangle$, 顶点与边的交替序列 $P = v_0 e_1 v_1 e_2 \dots e_l v_l$ 。

- 若对所有 i , 边 $e_i = (v_{i-1}, v_i)$, 则称 P 为 v_0 到 v_l 的**通路**。
- v_0 为**起点**, v_l 为**终点**, l 为通路的**长度**。

表示方法:

1. 按定义用顶点和边的交替序列: $P = v_0 e_1 v_1 e_2 \dots e_l v_l$
 2. 用边序列: $P = e_1 e_2 \dots e_l$
 3. 简单图中, 用顶点序列: $P = v_0 v_1 \dots v_l$
2. **回路 (Circuit):** 若起点与终点重合 ($v_0 = v_l$), 则称该通路为**回路**, 也称为**圈**。
- 长度为奇数的圈称为**奇圈**, 长度为偶数的圈称为**偶圈**。
3. **一些重要事实:**
- 在无向图中, 长度为 1 的圈由环构成, 长度为 2 的圈由两条重边构成。
 - 在无向简单图中, 所有圈的长度 ≥ 3 。
 - 在有向图中, 长度为 1 的圈由环构成。
 - 在有向简单图中, 所有圈的长度 ≥ 2 。
4. **分类与特征:**
- **简单通路/回路:** 通路中所有**边各异**。
 - **初级通路 (路径):** 简单通路中所有**顶点各异**。
 - **初级回路 (圈):** 简单回路中除端点外所有**顶点各异**。
 - **关系:** 初级通路/回路必是简单通路/回路, 但反之不一定成立。
 - **例子:**
 1. 初级通路: $v_1 v_2 v_3 v_4$ (所有顶点不同)
 2. 非初级的简单通路: $v_1 v_2 v_3 v_2 v_4$ (边各异但顶点重复)
 3. 初级回路: $v_1 v_2 v_3 v_1$ (除起点终点外顶点不同)
 4. 非初级的简单回路: $v_1 v_2 v_3 v_2 v_1$ (边各异但内部顶点重复)

5. **圈的长度限制:**
- 无向简单图中, 所有圈的长度 ≥ 3 。
 - 有向简单图中, 所有圈的长度 ≥ 2 。
6. **存在性定理:** 在无向图 G 中, 若任意顶点 v 的度数 $d(v) \geq 2$, 则 G 中一定存在圈。

证明: 不妨设 G 是简单图。如果 G 中存在圈, 得证。否则 G 中不存在圈, 则必然可在 G 中找到一条最长的路径 $L: v_0 v_1 \dots v_l$ 。考虑到 v_0 至少存在 2 个邻居, v_1 是其中之一, 假设 v_x 是另外一个邻居。

- 如果 v_x 在 L 中出现, 则 $v_x v_0 v_1 \dots v_x \dots v_l$ 成圈, 矛盾;
- 否则 $v_x v_0 v_1 \dots v_l$ 是一条更长的路径, 矛盾。

因此 G 中一定存在圈。

(2) 图的连通性

1. 无向图的连通性:

- **连通 (Connected):** 顶点 u 与 v 之间存在通路。
- **连通图:** 任意两点都连通的图。
- **连通支 (Connected Component):** 图的极大连通子图。 $p(G)$ 表示连通分支数。
- **判定:** 若简单图的边数 $m > \frac{1}{2}(n-1)(n-2)$, 则 G 是连通图。

2. 有向图的连通性:

- **可达 (Reachable):** u 到 v 有通路。
- **强连通:** 任意两点 u, v 相互可达。
- **单向连通:** 对于任意两点 u, v , u 可达 v 或 v 可达 u 。
- **弱连通:** 略去方向后的无向图是连通图。
- **强连通 $\rightarrow$ 单向连通 $\rightarrow$ 弱连通** (反之不真)。

3. 短程线与距离:

- **短程线:** 两点间长度最短的通路。
- **距离 $d(u, v)$:** 短程线的长度。不连通时规定为 ∞ 。
- 有向图距离不满足对称性 (即 $d < u, v >$ 不一定等于 $d < v, u >$)。

(3) 割点、割边与连通度

1. **割点 (Cut-vertex):** 若 $G - v$ 的连通分支数比 G 多, 则称 v 为割点。 **等价性质:**

- v 是割点当且仅当存在 u, w 使得所有 u 到 w 的路径都经过 v 。
- v 是割点当且仅当图 $V - v$ 可以划分为两个顶点集 U 和 W , 使得对任意顶点 $u \in U$ 和 $w \in W$, 顶点 v 都在每条道路 P_{uw} 上。

2. **割边 (Cut-edge/Bridge):** 若 $G - e$ 的连通分支数比 G 多, 则称 e 为割边。 **等价性质:**

- e 是割边当且仅当 e 不属于任何回路。
- e 是割边当且仅当存在 G 的顶点 u 和 w , 使 e 属于 u 和 w 的任何一条道路 $P_{\{uw\}}$ 。
- e 是割边当且仅当图 $G - e$ 可以划分为两个顶点集 U 和 W , 使得对任意顶点 $u \in U$ 和 $w \in W$, 道路 P_{uw} 都经过 e 。

3. **点连通度 $\kappa(G)$:** 使图不连通或只剩一个孤立点所需删除的**最小顶点数**。

- 若有割点, 则 $\kappa(G) = 1$; 不连通图为 0; K_n 为 $n - 1$ 。

4. **边连通度 $\lambda(G)$:** 使图不连通所需删除的**最小边数**。

5. **重要定理:** 对任意连通图 G , 有 $\kappa(G) \leq \lambda(G) \leq \delta(G)$ (其中 $\delta(G)$ 为最小度)。

证明:

第一步: 证明 $\lambda(G) \leq \delta(G)$

- 存在点 v 的度数是 $\delta(G)$
- 删掉与 v 相邻的所有边 (共 $\delta(G)$ 条) 可以使得 v 与其他节点不连通
- 因此 $\lambda(G) \leq \delta(G)$

第二步: 证明 $\kappa(G) \leq \lambda(G)$

- 设 $B = \{e_1, e_2, \dots, e_\lambda\}$ 为 G 的一最小边断集, $G - B$ 含两个不连通顶点子集 V_1, V_2
- 删除每条边在 V_1 中的端点, 则 B 中的边不复存在, 导致 V_1 和 V_2 不连通
- 极端情况下, V_1 被删空, 剩下的 V_2 不一定不连通
- **解决方法:** 留一个
 1. 删掉 $e_1, e_2, \dots, e_{\lambda-1}$ 在 V_1 中的端点
 2. 删掉 e_λ 在 V_2 中的端点
 3. 这样至多删掉 λ 个顶点使图不连通

• **更严谨的证明:**

1. 如果 G 有一个顶点 v 不在 B 中的任何边上, 设 C 是 $G - B$ 后包含 v 的那个连通支, 删掉 C 中的 B 的端点后 G 变得不连通, $\kappa(G) \leq \lambda(G)$
2. 如果不存在这样的 v , 即图中所有节点都关联 B 中的边, 此时假定 v 是其中任意一个节点, 那么 v 所有邻接节点必然都对应了 B 中不同的边, 因此 v 的邻接点数量不超过 λ , 删掉这些点将 v 和 $G - v$ 分开, 得证

• **推论:** 对于简单连通图 G , 有 $\kappa(G) \leq 2\frac{m}{n}$, 其中 m 为边数, n 为顶点数。

• **证明:** 由握手定理, $\sum d(v_i) = 2m$, 故 $n\delta(G) \leq 2m$, 即 $\delta(G) \leq 2\frac{m}{n}$ 。再由前述定理得 $\kappa(G) \leq \delta(G) \leq 2\frac{m}{n}$ 。

6. **点割集与边割集:**

- **点割集:** 若删除顶点集合 V' 后图的连通分支数增加, 且删除 V' 的任何真子集都不会增加连通分支数, 则称 V' 为点割集。
- **边割集:** 若删除边集合 E' 后图的连通分支数增加, 且删除 E' 的任何真子集都不会增加连通分支数, 则称 E' 为边割集。
- **点断集:** 使连通图变得不连通或只剩一个孤立节点的顶点集合。
- **边断集:** 使连通图变得不连通的边集合。

(4) 块 (Block)

• **定义:** 图中极大的没有割点的连通子图。

定理 2.1.3: 设 G 是至少有 3 个顶点的连通图, 则下述性质等价:

1. G 是一个块。
2. G 的任何两个顶点同属某一初级回路。
3. G 的任何一个顶点和任何一条边同属某个初级回路。
4. G 的任何两条边同属某个初级回路。
5. 给定两个点 u, v 和一条边 e , 存在一条包含 e 的初级道路 P_{uv} 。
6. 对 G 的任意三个不同的顶点, 存在一条包含它们的初级道路。
7. 对 G 的任意三个不同的顶点, 存在一条只包含其中两点而不含第三点的道路。

(5) 复杂网络简介

- **小世界现象**: 大部分节点彼此不相连, 但绝大部分节点之间经过少数几步就可到达。
 1. 特征: 高集聚系数和低平均路径长度
 2. 六度分隔理论: 任意两人之间“距离”约为 6
 3. 贝肯数、埃尔德什数
- **无尺度特征**: 大部分节点只和很少节点连接, 极少数节点与非常多节点连接。
 1. 度分布: 幂律分布 $P(d = k) \sim k^{-\gamma}$, γ 通常在 2 到 3 之间。

(6) 道路与回路的判定方法

1. **代数方法——邻接矩阵幂**:
 - 设 A 是图 G 的邻接矩阵, 则 A^l 中 (i, j) 位置元素为从顶点 v_i 到 v_j 长度为 l 的路的数目。
 - 证明: 对 l 归纳, $A^l = A^{l-1} * A$ 。
2. **可达矩阵**:
 - $P(G) = A + A^2 + \dots + A^n$ 或 $P = A \vee A^2 \vee \dots \vee A^n$ (逻辑运算)
 - 若只关心可达性, 可用逻辑运算法, 复杂度 $O(n^4)$ 。
3. **Warshall 算法**:

```
P = [a_(i j)]_(n times n)
for k = 1 to n:
  for i = 1 to n:
    for j = 1 to n:
      p_(i j) = p_(i j) or (p_(i k) and p_(k j))
```

- 复杂度: $O(n^3)$
 - 原理: $p_{ij}^{(k)} = p_{ij}^{(k-1)} \vee (p_{ik}^{(k-1)} \wedge p_{kj}^{(k-1)})$
 - $p_{ij}^{(k)} = 1$ 当且仅当顶点集 $\{v_i, v_1, v_2, \dots, v_k, v_j\}$ 之间有 v_i 到 v_j 的道路。
4. **所有节点对最短路 (Floyd 算法)** :

```
P = [a_(i j)]_(n times n) // 邻接=1, 不邻接=$infinity$
for k = 1 to n:
  for i = 1 to n:
    for j = 1 to n:
      P_(i j) = min(P_(i j), P_(i k) + P_(k j))
```

5. 应用举例——序列整除问题:

- 问题: 任意给定 $(n+1)^2$ 项的递增自然数列, 证明以下结论必有一条成立: (1) 存在 $n+1$ 项的子列, 使任一项能整除此子列中它后面的每一项; (2) 存在 $n+1$ 项的子列, 使此子列中任一项不能整除此子列中它后面的任何一项。
- 解法: 建立有向图模型, 顶点为数列元素, 若 a_i 整除 a_j 则有边 (a_i, a_j) 。
- 对每个顶点 v_i , 定义 $l(v_i)$ 为以 v_i 为起点的最长有向路径长度。
- 若存在 $l(v_i) = n$, 则结论(1)成立。
- 否则所有 $l(v_i) \leq n-1$, 由鸽巢原理, 存在 $n+1$ 个顶点具有相同的 $l(v_i)$ 值, 这些顶点互不相邻, 结论(2)成立。

(7) 搜索算法

1. 搜索基本概念:

- 基本依据：邻接关系
- 原则：每个节点只访问 1 次，迭代到没有新的可访问节点为止。
- 节点状态：
 1. Invisible：还未被发现的节点
 2. Visible：已被发现（未访问，访问过前驱节点）
 3. Visited：已访问过
- 通用搜索框架：

```
visible = ${v_0}$; // $v_0$ 是起始节点
visited = $emptyset$;
while |visible| > 0:
    v = fetch(visible); // 从可见节点中取出一个
    visit(v); // 访问该节点
    visited = visited union ${v}$;
    visible = (visible union  $\Gamma^+(v)$ ) setminus visited;
```

- **fetch 策略决定搜索方式。**

3. 宽度优先搜索 (BFS):

- 策略：自上往下，逐层选择；越靠上（先发现）越优先。
- 数据结构：**队列**（先发现，先访问）。
- 特点：找到的是**最短路**（步数最少）。

```
BFS(v):
    flag[v] = 1; // 1=visible, 2=visited, 0=invisible
    Q = [v]; // 队列初始化
    while Q 非空:
        u = DEQUEUE(Q);
        for 邻接于 u 的所有顶点 w:
            if flag[w] = 0:
                ENQUEUE(w, Q);
                flag[w] = 1;
                pred[w] = u; // 记录前驱以还原路径
        flag[u] = 2;
```

3. 深度优先搜索 (DFS):

- 策略：优先选择新发现的节点扩展；越靠下（晚发现）越优先。
- 数据结构：**栈**（后发现，先访问）。
- 特点：尽量向下搜索，只有碰壁才回头。

```
DFS(v):
    visited[v] = 1;
    for 邻接于 v 的每个顶点 w:
        if visited[w] = 0:
            DFS(w); // 递归
```

4. 应用举例——过河问题:

- 人带狼、羊、白菜过河，每次只能带一样，狼羊、羊菜不能独处。
- 解法：建立图模型，16 种状态中排除非法状态剩 10 种合法状态，求从初始到目标的路径。

四、特殊道路与回路问题 2026-03-17

(1) 欧拉道路与回路

基本概念与定义

哥尼斯堡七桥问题 (1736)：图论的第一篇论文。问题转化为：从图中任一点出发，通过每条边且仅一次，最后返回到出发点。

基本术语：

- **欧拉通路**：经过所有顶点，且每条边恰好经过一次的通路
- **欧拉回路**：经过所有顶点，且每条边恰好经过一次的回路
- **欧拉图**：有欧拉回路的图
- **半欧拉图**：有欧拉通路而无欧拉回路的图

说明：

1. 上述定义对无向图和有向图都适用
2. 欧拉通路是简单通路，欧拉回路是简单回路
3. 环不影响图的欧拉性

欧拉图判别定理

定理 2.3.1： 无向连通图 G 有欧拉回路的充要条件是各顶点的度都是偶数。

证明：

必要性： 已知存在欧拉回路，要证明度都是偶数。

- 欧拉回路经过每边一次且仅一次
- 沿该回路进入某点后，必定经由另一条边出去
- 因此，各点的度都是偶数

充分性（构造法）： 已知连通图各顶点的度都是偶数，则必存在欧拉回路。

步骤：

1. 从任意点 v_0 出发，构造 G 的一条简单回路 C
 - 由于 v_i 的度为偶，所以不可能停留在某点 $v_i \in V - \{v_0\}$ 上而不能继续向前构造
 - 由于 G 是有穷图，因此最终一定能够回到 v_0 ，构成简单回路 C
2. 若 C 包含了 G 中的所有边，它即是 G 的欧拉回路
3. 否则，从 G 中删去 C 的各边，得到 G_1 。显然 G_1 中每点的度仍然是偶数
4. 此时， G_1 中一定存在度非 0 的顶点 v_i ，它同时还是回路 C 经过的顶点（否则 G 是非连通图）
5. 在 v_i 所在的 G_1 的连通支中，同理可构造简单回路 C' ，令 $C = C \cup C'$ ，得到包含边数比原来更多的简单回路
6. 继续上述构造过程，最终该简单回路必是包含了所有边，即构造出了欧拉回路

构造欧拉回路的算法

算法步骤：

1. 任选节点 v 作为起点
2. 以 v 为起点找简单回路 E_1
3. 若 E_1 包含所有边，输出 E_1 ，结束
4. 否则，在 $G - E_1$ 中找一个简单回路 E_2 ，使 E_1 与 E_2 至少有一个公共点
5. 以某公共点为起、末点，对 $E_1 \cup E_2$ 中的边重新排序得新的简单回路 C
6. $E_1 := C$ ，转步骤 3

例题： 求右图中欧拉回路。（图见 slides P7）

解答：

1. 选以 1 为起点的连通回路 E_1 ：1261
2. E_1 不包含所有边
3. 在 $G - E_1$ 中找新回路 E_2 ：6356（6 是 E_1 与 E_2 的公共点）
4. 以公共点 6 为起点，对 $E_1 \cup E_2$ 中的边排序： $C = 6356126$
5. $E_1 := C$ ； E_1 不包含所有边
6. 在 $G - E_1$ 中找新连通回路 E_2 ：52345（5 是 E_1 与 E_2 的公共点）
7. 以公共点 5 为起点，对 $E_1 \cup E_2$ 中的边排序： $C = 52345612635$
8. $E_1 := C$ ； E_1 包含所有边
9. 输出 $E_1 = 52345612635$

欧拉道路判别定理

推论 2.3.1： 若无向连通图 G 中只有两个奇顶点，则 G 存在欧拉道路。

证明：

- 设这两个奇顶点是 v_i, v_j
- 在图 G 中加入一条边 (v_i, v_j) ，则所有的顶点的度都为偶，此时其中必然存在一条欧拉回路
- 然后将边 (v_i, v_j) 去掉，可得从 v_i 到 v_j 的欧拉道路

k 笔画问题

定理 2.3.2： 设连通图中有 k 个度为奇数的顶点。证明 $E(G)$ 可以划分成 $\frac{k}{2}$ 条简单道路。

证明（构造法）：

1. 由图的性质可得， k 是偶数
2. 在这 k 个顶点中两两配对，增添互不相邻的 $\frac{k}{2}$ 条边，得到 G'
3. G' 中每点的度都是偶数，由定理， G' 中有欧拉回路 C
4. 在 C 中删去这 $\frac{k}{2}$ 条边，便得到了 $\frac{k}{2}$ 条简单道路，它们包含了原图 G 中的所有边
5. 即这 $\frac{k}{2}$ 条简单道路就是 $E(G)$ 的一个划分

推论 2.3.2： 设图 G 是连通的，并且恰有 $2k$ 个奇点，那么图形 G 可用 k 笔不重复画成，并且至少要用 k 笔画成。

有向图的欧拉道路与回路

推论 2.3.3： 若有向连通图 G 中各个节点的正负度相等（入度等于出度），则 G 中存在有向欧拉回路。

推论 2.3.4： 若有向连通图 G 中只有两个节点的正负度不相等，而且其中一个入度比出度多 1，另一个入度比出度少 1，则 G 中存在有向欧拉通路。

例题：（见 slides P22-24）

给定有向图，判断是否有欧拉回路/通路：

- 计算各点入度和出度
- 若所有点入度=出度，有欧拉回路
- 若恰有两点入度!=出度，一个多 1 一个少 1，有欧拉通路
- 否则无欧拉通路/回路

应用：计算机编码盘设计

问题： 设有编码盘其表面被等分成 $2^4 = 16$ 个部分，每一部分分别用绝缘体或导体组成。问编码盘上 16 个部分怎样安排导体及绝缘体，才能使编码盘每旋转一个部分，四个触点能得到一组不同的四位二进制数信息？

观察与建模：

- 每次旋转时，输出中有三位不变，如： $abc \rightarrow abcy$
- **节点：** 四位数字的后三位，八种组合 = 八个节点
- **边：** 每次旋转可以从一个节点到另一个节点： $abc \rightarrow abcy$ ，其中 y 有两种可能

构造有向连通图：

- 8 个节点，16 条边（每个节点出度=入度=2）
- 包括了 16 种输出
- 每个节点的度都是偶数，因此存在欧拉回路

解法： 任何一条欧拉回路都是一种可行方案

- 例如： $e_0 \rightarrow e_1 \rightarrow e_2 \rightarrow e_5 \rightarrow e_{10} \rightarrow e_4 \rightarrow e_9 \rightarrow e_3 \rightarrow e_6 \rightarrow e_{13} \rightarrow e_{11} \rightarrow e_7 \rightarrow e_{15} \rightarrow e_{14} \rightarrow e_{12} \rightarrow e_8$
- 对应：0000101001101111...

(2) 哈密顿道路与回路

基本概念

哈密顿周游世界问题（Hamilton 1856 年）：游览十二面体上的 20 个顶点（世界 20 个城市），30 条棱边对应 20 个城市间的交通路线，怎样能够每个城市只经过一次并回到出发点。

基本术语：

- **哈密顿通路（H-通路）：** 连通图的一条经过所有顶点一次且仅一次的通路
- **哈密顿回路（H-回路）：** 连通图中经过所有顶点一次且仅一次的回路
- **哈密顿图：** 具有哈密顿回路的图
- **半哈密顿图：** 具有哈密顿通路的图

说明:

1. 哈密顿通路是初级通路
2. 哈密顿回路是初级回路
3. 环与重边不影响图的哈密顿性, 故只考虑简单图

哈密顿图的实质: 能将图中所有的顶点排在同一个圈中。

哈密顿图与欧拉图的关系

一个图可以:

- 既是哈密顿图也是欧拉图
- 既是哈密顿图也是半欧拉图
- 是哈密顿图但不是欧拉图也不是半欧拉图
- 既是半哈密顿图也是欧拉图
- 不是半哈密顿图也不是哈密顿图但是欧拉图
- 既是半哈密顿图也是半欧拉图

哈密顿回路判定——基本性质

说明:

1. 目前为止, 还没有找到简单的充要条件来判断哈密顿回路的存在性
2. 但已经有许多定理对其存在性给出了充分条件
3. 某些性质可用来判明一个图没有哈密顿回路:
 - 带有 1 度顶点的图没有哈密顿回路
 - 若图中有 2 度顶点, 则关联这个顶点两条边属于任何哈密顿回路
 - 每个顶点只能有两条关联的边在哈密顿回路内
 - 哈密顿回路不能包含更小的回路

简单结论: 至少有 3 个点的完全图是 H 图。

哈密顿回路判定——必要条件

定理: 若图 $G = \langle V, E \rangle$ 有 H 回路, 则对 V 的任何非空子集 S , 均有 $p(G - S) \leq |S|$, 其中 $p(G - S)$ 是从 G 中删去 S 中所有节点及与这些节点关联的边所得到的子图的连通分支数。

证明: 设 C 是图 G 的一条 H 回路, 则对于 V 的任何非空子集 S :

- 在 C 中删去 S 中任意一个节点 v_1 后, 则 $C - v_1$ 仍是连通的路
- 若再删去 S 中的另一个节点 v_2 , 则 $p(C - v_1 - v_2) \leq 2$
- 若删去 S 中的 k 个节点, 则 $p \leq k$
- 即 $p(C - v_1 - v_2 - \dots - v_k) \leq |S|$
- 因为 C 是 H 回路, 所以它包含了 G 的所有节点, 即 C 是 G 的生成子图
- 所以 $C - S$ 也是 $G - S$ 的生成子图
- 故 $p(G - S) \leq p(C - S) \leq |S|$

推论 1: 若图 $G = \langle V, E \rangle$ 有 H-道路, 则对 V 的任何非空子集 S , 均有 $p(G - S) \leq |S| + 1$ 。

证明: 设 P 是图 G 的一条起于 u 终于 v 的 H-路:

- 令 $G_1 = G + (u, v)$, 显然 G_1 是哈密顿图
- $p(G_1 - S) \leq |S|$
- $p(G - S) = p(G_1 - S - (u, v)) \leq p(G_1 - S) + 1 \leq |S| + 1$

推论 2: 有割点的图不是哈密顿图。

证明: 设 v 为割点, 则 $p(G - v) \geq 2 > |\{v\}| = 1$, 根据定理, 得证。

推论 3: 若图 $G = \langle V_1, V_2, E \rangle$ 为二分图且有哈密顿回路, 则 $|V_1| = |V_2|$ 。

注意: 定理中的条件是哈密顿图的必要条件, 但不是充分条件。

- **反例:** Petersen 图满足必要条件, 但不是哈密顿图

应用: 1/4 国际象棋盘(4 times 4 方格)上的跳马问题: 马是否能恰好经过每一个方格一次后回到原处?

解:

1. 每个方格看作一个顶点, 2 个顶点之间有边当且仅当马可以从一个方格跳到另一个方格, 得到 16 阶图 G

2. 取 $V_1 = \{a, b, c, d\}$ (某四个特定方格), 则 $p(G - V_1) = 6 > |V_1| = 4$
3. 由定理, 图中无哈密顿回路, 故问题无解

哈密顿道路判定——充分条件

定理 2.4.1 (Ore 定理——道路形式): 若简单图 G 中任两点 u, v , 恒有 $d(u) + d(v) \geq n - 1$, 则 G 中存在哈密顿道路。

证明 (构造法):

第一步: 证明 G 连通

- 反证法: 假设 G 不连通, 则 G 至少有两个连通分支, 设为 H_1, H_2
- 取 $u \in H_1, v \in H_2$
- 则 $d(u) \leq |H_1| - 1, d(v) \leq |H_2| - 1$
- 所以 $d(u) + d(v) \leq |H_1| - 1 + |H_2| - 1 \leq n - 2$, 与已知矛盾
- 所以 G 连通

第二步: 构造连通图 G 中的 H 道路

- 设 P 为 G 的包含 l 个节点的极长初级道路, $P = (v_{i_1}, v_{i_2}, \dots, v_{i_l})$
- 则与 v_{i_1} 和 v_{i_l} 相邻的点都在 P 上
- 若 $l = n$, 则 P 为 H 道路, 得证
- 若 $l < n$, 要证明 G 中一定存在经过节点 $v_{i_1}, v_{i_2}, \dots, v_{i_l}$ 的初级回路 C

证明存在回路 C :

- 假设道路中的节点不形成初级回路
- 设 $(v_{i_1}, v_{i_p}) \in E(G)$, 则不能有 $(v_{i_{p-1}}, v_{i_l}) \in E(G)$, 否则删除 $(v_{i_{p-1}}, v_{i_p})$, 形成一个回路
- 设 $d(v_{i_1}) = k$, 则 $d(v_{i_l}) \leq l - k - 1$
- 则 $d(v_{i_1}) + d(v_{i_l}) \leq l - 1 < n - 1$, 与已知矛盾
- 因此存在回路 C

构造更长的道路:

- 设 $C = (v_{i_1}, v_{i_2}, \dots, v_{i_l}, v_{i_1})$, 由于 G 连通, 故存在 C 之外的节点 v_t , 与 C 中的某点 v_{i_q} 相邻
- 可构造长为 $l + 1$ 的初级道路 $P' = (v_t, v_{i_q}, v_{i_{q+1}}, \dots, v_{i_l}, v_{i_1}, \dots, v_{i_{q-1}})$
- 依次构造, 直至 $l = n$, 得到 H 道路

扩大路径法: 包含 L 个节点的极长初级道路, 首尾节点度数和 $\geq L$ 时 (也就是 $> L - 1$ 时), 必可构造回路。

推论 2.4.1 (Ore 定理——回路形式): 若简单图 G 中任两点 u, v , 恒有 $d(u) + d(v) \geq n$, 则 G 中存在哈密顿回路。

证明:

- 由定理可知, G 中存在哈密顿道路 H , 设 H 为 $v_1, v_2, \dots, v_n$
- 假设不存在 H 回路
- 设 $d(v_1) = k$, 则 $d(v_n) \leq n - k - 1$ (原因见定理 2.4.1)
- 则 $d(v_n) + d(v_1) < n$, 与已知矛盾
- 因此存在初级回路, 即 H 回路

推论 2.4.2 (Dirac 定理): 若简单图 G 中每个节点的度都大于等于 $n/2$, 则 G 中存在哈密顿回路。

注意: 这是充分条件, 不是必要条件。有些图有哈密顿回路, 但既不满足 Dirac 定理, 也不满足 Ore 定理。

应用例题: 某次国际会议 8 人参加, 已知每人至少与其余 7 人中的 4 人有共同语言, 问服务员能否将他们安排在同一张圆桌就座, 使得每个人都能与两边的人交谈?

解:

- 作无向图 $G = \langle V, E \rangle$, 其中 $V = \{v \mid v \text{ 为与会者}\}$, $E = \{(u, v) \mid u, v \in V, u \text{ 与 } v \text{ 有共同语言}\}$
- G 为简单图。根据条件, $\forall v \in V, d(v) \geq 4$
- 于是, $\forall u, v \in V$, 有 $d(u) + d(v) \geq 8$
- 由 Ore 定理可知 G 为哈密顿图
- 服务员在 G 中找一条哈密顿回路 C , 按 C 中相邻关系安排座位即可

闭合图与判定定理

引理 2.4.1: 简单图 G 中 v_i 和 v_j 是不相邻节点, 满足 $d(v_i) + d(v_j) \geq n$, 则 G 存在 H 回路的充要条件是 $G + (v_i, v_j)$ 有 H 回路。

证明:

- **必要性:** 显然
- **充分性:** 假设 G 不存在 H 回路
 - $G + (v_i, v_j)$ 的 H 回路定有边经过 (v_i, v_j) , 删去此边, G 中存在以 v_i, v_j 为端点的 H 道路
 - 根据假设, 有 $d(v_i) + d(v_j) < n$, 与已知矛盾
 - 因此充分性满足

定义: 闭合图: 若 v_i 和 v_j 是简单图 G 的不相邻节点, 满足 $d(v_i) + d(v_j) \geq n$, 则令 $G' = G + (v_i, v_j)$, 对 G' 重复上述过程, 直到不再有这样的节点对。最终得到的图称为 G 的闭合图, 记为 $C(G)$ 。

引理 2.4.2: 简单图 G 的闭合图是唯一的。

证明: 设 $C_1(G)$ 和 $C_2(G)$ 是 G 的两个闭合图, $L_1 = \{e_1, e_2, \dots, e_r\}$, $L_2 = \{a_1, a_2, \dots, a_s\}$ 是 $C_1(G)$ 和 $C_2(G)$ 中新加入的边集合。

需证 $L_1 = L_2$:

- 假设 $L_1 \neq L_2$, 不妨设 $e_{i+1} = (u, v) \in L_1$ 是构造时第一条不属于 L_2 的边
- 令 $H = G \cup \{e_1, e_2, \dots, e_i\}$
- 由于构造 L_1 时加入了 e_{i+1} , 则有 $d(u) + d(v) \geq n$
- 而且 $C_2(G)$ 中包含 H , 因此 $C_2(G)$ 中 $d(u) + d(v) \geq n$ 成立
- 但是 (u, v) 不属于 $C_2(G)$, 与 $C_2(G)$ 是 G 的闭合图矛盾
- 故 $L_1 = L_2$

定理 2.4.2: 简单图 G 存在哈密顿回路的充要条件是其闭合图存在哈密顿回路。

证明: 设 $C(G) = G \cup L$, $L = \{e_1, e_2, \dots, e_r\}$, 由引理: G 有 H 回路 $\leftrightarrow G + e_1$ 有 H 回路 $\leftrightarrow \dots \leftrightarrow G \cup L$ 有 H 回路

由于 $C(G)$ 唯一, 定理得证。

推论 2.4.3: 若简单图 G ($n > 2$) 的闭合图是完全图, 则 G 有哈密顿回路。

哈密顿图判定方法总结

判断步骤:

1. 不满足必要条件 $\rightarrow$ 没有 H 回路
2. 满足充分条件 $\rightarrow$ 有 H 回路
3. 其他 $\rightarrow$ 具体问题具体分析
 - 构造闭合图
 - 搜索出一条哈密顿回路

(3) 欧拉回路与哈密顿回路的比较

比较项	欧拉回路	哈密顿回路
回路类型	简单回路	初级回路
定义	过所有边一次且仅一次	过所有点一次且仅一次
判定方法	有充要条件, 简单	无充要条件, 复杂

(4) 判定定理汇总

欧拉回(道)路判定的充要条件

- **无向连通图 G 有欧拉回路的充要条件是各顶点的度都是偶数**
- 若无向连通图 G 中只有两个奇顶点, 则 G 存在欧拉道路
- 若有向连通图 G 中各个节点的正负度相等, 则 G 中存在有向欧拉回路
- 若有向连通图 G 中只有两个节点的正负度不相等, 而且其中一个入度比出度多 1, 另一个入度比出度少 1, 则 G 中存在有向欧拉通路

哈密顿回(道)路判定的必要条件

- 若图 $G = \langle V, E \rangle$ 有 H-回路, 则对 V 的任何非空子集 S , 均有 $p(G - S) \leq |S|$
- 若图 $G = \langle V, E \rangle$ 有 H-道路, 则对 V 的任何非空子集 S , 均有 $p(G - S) \leq |S| + 1$
- 有割点的图不是哈密顿图
- 若图 $G = \langle V_1, V_2, E \rangle$ 为二分图且有哈密顿回路, 则 $|V_1| = |V_2|$

哈密顿回(道)路判定的充分条件

- **Ore 定理 (道路)**: 若简单图 G 中任两点 u, v , 恒有 $d(u) + d(v) \geq n - 1$, 则 G 中存在哈密顿道路
- **Ore 定理 (回路)**: 若简单图 G 中任两点 u, v , 恒有 $d(u) + d(v) \geq n$, 则 G 中存在哈密顿回路
- **Dirac 定理**: 若简单图 G 中任意一点 v , 有 $d(v) \geq n/2$, 则 G 中存在哈密顿回路
- 若简单图 G ($n > 2$) 的闭图是完全图, 则 G 有哈密顿回路

哈密顿回路充要条件

简单图 G 存在哈密顿回路的充要条件 简单图 G 中 v_i, v_j 不相邻节点, 满足 $d(v_i) + d(v_j) \geq n$, 则 G 存在 H 回路的充要条件是 $G + (v_i, v_j)$ 有 H 回路。

证明: 必要性显然。

充分性: 假设 G 不存在 H 回路。 $G + (v_i, v_j)$ 的 H 回路定有边经过 (v_i, v_j) , 删去此边, G 中存在以 v_i, v_j 为端点的 H 道路。根据假设, 有 $d(v_i) + d(v_j) < n$, 与已知矛盾。因此充分性满足。

(5) 旅行商问题与分支定界法

旅行商问题 (TSP): 给定 n 个城市和它们之间的距离, 求访问每个城市恰好一次并回到起点的最短回路。

如果纯枚举, 时间复杂度为 $O(n!)$ 。那么我们要剪枝!

一句话简单总结: 对于 n 个点, 我们 DFS 深度为 n 。当在某同一层发现长度已经大于等于当前最优解时, 我们剪枝。否则一直走到最底层, 看是否构成回路, 并更新最优解。好像听起来很简单。但是正是因为听起来简单, 所以复杂度依旧很高。

实际上上述依旧很像纯暴力。但是下面说了初始下界.....

实际上完善的分支定界需要下界和上界。

分支定界法:

- 基本思想: 系统地枚举所有可能解, 但在搜索过程中剪枝不可能产生最优解的分支
- 步骤:
 1. 计算初始下界 (如最小生成树权重的 2 倍)
 2. 选择一个城市作为根节点开始分支
 3. 对每个分支计算下界, 如果下界大于当前最优解, 则剪枝
 4. 继续分支直到找到最优解或所有分支都被剪枝

“便宜”算法 (近似解法)

基本思想: 贪心策略——将问题求解看作一系列选择, 每次选择当前状态下的最好选择 (局部最优解), 希望逐步达到整体最优解。

适用前提:

- 整体最优解可通过一系列局部最优解达到
- 每次选择可依赖以前的选择, 但不能依赖后面的选择
- 最优子结构: 问题的整体最优解中包含子问题的最优解

算法原理:

- T 是一个不断扩充的初级回路
- 最初 T 是一个自环 $(1, 1)$
- 每次找一个与 T 最近的节点 j , 将 j 插入 T 中
- 具体插入位置根据增量大小决定: 若 $w(j, t) + w(j, t_1) - w(t, t_1) \leq w(j, t) + w(j, t_2) - w(t, t_2)$, 则插到 t 与 t_1 之间; 否则插到 t 与 t_2 之间

算法步骤:

1. 设 $w(i, j)$ 为边权值, 对任意 i , $w(i, i) = 0$; 置 $S = \{2, 3, \dots, n\}$, 序列 $T = (1, 1)$
2. 在 S 中, 找到使 $w(i, l)$ 最小的 $j \in S$ 和 $t \in T$

3. 对回路 T 中的边 $(t, t_1), (t, t_2)$, 根据增量大小决定插入位置

4. 令 $S = S - \{j\}$, 若 $S = \emptyset$ 则结束, 否则转步骤 2

复杂度: 朴素实现 $O(n^3)$; 借助辅助数组优化后 $O(n^2)$

辅助数组两个: 一个用来存当前最短距离, 一个存对应的点。

与分支定界法的比较:

- **效率:** 便宜算法 $O(n^2)$ 远优于分支定界法 $O(n!)$
- **质量:** 分支定界法得到精确最优解, 便宜算法得到近似解
- **近似比:** 便宜算法只能保证 $\frac{T}{Q} < 2$ (T 为近似解, Q 为最优解)
- **实际效果:** 实际中便宜算法与最优解通常非常接近

(6) 最短路径

基本定义

问题背景: 最短路径问题是网络理论中应用最广的问题之一。许多优化问题可以使用这个模型, 如设备更新、管道铺设、线路安排、厂区布局等。

一般提法: 设 $G = (V, E)$ 为连通图, 图中各边 v_i, v_j 有权 w_{ij} ($w_{ij} = \infty$ 表示 v_i, v_j 间无边), v_s, v_t 为图中任意两点, 求一条道路 μ , 使它是从 v_s 到 v_t 的所有道路中总权最小的道路:

$$L(\mu) = \sum_{(v_i, v_j) \in \mu} w_{ij}$$

最小。

路径长度定义: v_1 到 v_i 的一条路径 $P(i)$ 长度记为 $\pi(i)$:

$$\pi(i) = \sum_{e \in P(i)} w(e)$$

重要引理

定理: 正权图中两点间的最短路径一定是初级道路。

引理 2.6.1: 正权图中任一条最短路径长度大于其局部路径长度。

引理 2.6.2: 正权图中若 $P(i)$ 是 v_1 到 v_i 的最短路, 且 $v_j \in P(i)$, 则 $P(j)$ 也是 v_1 到 v_j 最短路。

证明: 反证法。若 $P(j)$ 不是最短路, 则存在更短的路径 $P'(j)$ 。用 $P'(j)$ 替换 $P(i)$ 中 v_1 到 v_j 的部分, 得到更短的 v_1 到 v_i 的路径, 与 $P(i)$ 是最短路矛盾。

Dijkstra 算法

基本思想 (1959 年提出): 按路径长度递增的次序产生最短路径。

核心概念:

- 设置集合 P , 存放已求出最短路径的顶点
- $T = V - P$ 是尚未确定最短路径的顶点集合
- 每个顶点对应一个距离值 Π :
 1. 集合 P 中顶点: 从 v_1 到该顶点的最短路径长度
 2. 集合 T 中顶点: 从 v_1 到该顶点的只包含集合 P 中顶点为中间顶点的最短路径长度

关键观察: T 里面最小元素一定是最短路, 可以将其转移到 P 中, 同时更新 T 中其他元素的距离。

更新公式: 对于当前 T 中 π 最小的节点 v , 检查其每一个后继 u :

$$\Pi(u) = \min\{\Pi(u), \Pi(v) + w(v, u)\}$$

算法步骤:

1. **初始化:** 集合 P 中只有顶点 v_1 , $\Pi(v_1) = 0$; 集合 $T = V - P$ 中顶点 v_i 的距离值为边 (v_1, v_i) 的权 w_{1i} , 若无直接相连则为 ∞
2. **贪心选择:** 在集合 T 中选择距离值 π 最小的顶点 j 加入集合 P , $P = P \cup \{j\}$, $T = T - \{j\}$
3. **修正:** 对集合 T 中 j 的后继节点 i 的距离值进行修正: $\Pi(i) = \min(\Pi(i), \Pi(j) + w_{ji})$
4. 重复步骤 2 和 3, 直到 T 为空

复杂度: $O(n^2)$

存储最短路径: 增设 n 维向量 Q , 初值为 1。修正时, 若新确定了最短路径的顶点 j 加入后使距离值更小, 则修改 $\Pi(i)$ 并将 j 赋给 $Q(i)$ 。最终可通过 Q 反向追踪得到路径。

其他最短路径算法

BFS (边权为 1): 当边权为 1 时, 可用广度优先搜索求最短路径, 复杂度 $O(m)$, m 为边数。

Ford 算法 (边权为任意实数):

- 适用于存在负权边的情况
- 基本思想: 根据路径长度从 0 到 $|V| - 1$, 逐步迭代得到最短路径
- 算法步骤:
 1. 赋初值 $\pi(1) = 0, \pi(i) = \infty \ (i = 2, 3, \dots, n)$
 2. 更新 $\pi(i) = \min\{\pi(i), \min\{\pi(j) + w_{ji} \mid j \in \Gamma^-(i)\}\}$, 其中 $\Gamma^-(i)$ 为 i 的直接前驱集
 3. 若所有 $\pi(i)$ 都无变化, 停止; 否则转步骤 2
- 复杂度: $O(mn)$
- 注意: 若有负长回路, 算法无法终止

Floyd 算法 (任意两点间最短路径, 1962 年提出):

- 基本思想: 逐个节点试探法
- 设 $A = (a_{ij})_{n \times n}$ 为赋权图的权矩阵
- 步骤:
 1. 赋初值: 对所有 $i, j, d_{ij} = a_{ij}, k = 1$
 2. 更新 d_{ij} : 对所有 i, j , 若 $d_{ik} + d_{kj} < d_{ij}$, 则令 $d_{ij} = d_{ik} + d_{kj}$
 3. 若 $k = n$ 终止; 否则 $k = k + 1$, 转步骤 2
- 复杂度: $O(n^3)$

最短路径算法总结

情况	边权为正数	边权为 1	边权为任意实数
某点到其他点	Dijkstra	BFS	Ford (负长回路不终止)
任意两点之间	Floyd	Floyd	Floyd (负长回路不正确)

Dijkstra 算法 (单源最短路径, 非负权):

```
Dijkstra(s):
  for each vertex v:
    dist[v] = infinity
    prev[v] = undefined
  dist[s] = 0
  Q = all vertices
  while Q is not empty:
    u = vertex in Q with min dist[u]
    remove u from Q
    for each neighbor v of u:
      alt = dist[u] + weight(u, v)
      if alt < dist[v]:
        dist[v] = alt
        prev[v] = u
```

Dloyd-Warshall 算法 (所有点对最短路径):

```
for k = 1 to n:
  for i = 1 to n:
    for j = 1 to n:
      dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
```

Ford 算法 (单源最短路径, 允许负权边):

```

Ford(s):
    for each vertex v:
        dist[v] = infinity
        prev[v] = undefined
    dist[s] = 0

    for i = 1 to |V| - 1: // 最多松弛 |V|-1 次
        for each edge (u, v) in E:
            if dist[u] + weight(u, v) < dist[v]:
                dist[v] = dist[u] + weight(u, v)
                prev[v] = u

    // 检测负权回路
    for each edge (u, v) in E:
        if dist[u] + weight(u, v) < dist[v]:
            return "存在负权回路, 无法终止"

    return dist, prev

```

(7) 关键路径

引理与定理 2.7.1

引理 2.7.1: 若有向图 G 无有向回路, 则一定存在入度为 0 的点及出度为 0 的节点。

证明: 在 G 中构造一条极长的有向道路 $P = (v_{i_1}, v_{i_2}, \dots, v_{i_l})$, 则 $d^-(v_{i_1}) = 0$, $d^+(v_{i_l}) = 0$ 。

否则如果 $d^-(v_{i_1}) \neq 0$, 则一定有边 $(v_k, v_{i_1}) \in E(G)$, 此时:

- 若 $v_k \in P$, 则 G 存在有向回路, 与已知矛盾;
- 若 v_k 不在 P 中, 则 P 不是极长道路, 与假设矛盾。

因此 $d^-(v_{i_1}) = 0$, 同理可证 $d^+(v_{i_l}) = 0$ 。

定理 2.7.1: 若有向图 G 无有向回路, 则可以将 G 的节点重新编号为 $v'_1, v'_2, \dots, v'_n$, 使得对任意的边 $v'_i v'_j \in E(G)$, 都有 $i < j$ 。

证明 (构造法): 构造节点序列:

1. 由引理 2.7.1, G 中存在入度为 0 的点
2. 取一个这样的点编号为 v'_1 , 令 $G \rightarrow G - v'_1$, 得到的图仍然没有回路
3. 再取一个入度为 0 的点编号为 v'_2
4. 重复这个过程, 直到所有点都编号为止

这时所有的编号都满足定理的条件, 图中所有边都是从编号小的节点指向编号大的节点。

拓扑排序

定义: 设 $G = (V, E)$ 是一个具有 n 个顶点的有向图, V 中顶点序列 $v_1, v_2, \dots, v_n$ 称为一个**拓扑序列**, 当且仅当该顶点序列满足下列条件: 若 $\langle v_i, v_j \rangle$ 是图中的边 (即从顶点 v_i 到 v_j 有一条路径), 则在序列中顶点 v_i 必须排在顶点 v_j 之前。

在一个有向图中找一个拓扑序列的过程称为**拓扑排序**。

例 2.7.1: 计算机专业的学生必须完成一系列规定的基础课和专业课才能毕业, 假设这些课程的名称与相应代号有如下关系:

代号	课程名称	先修课程
C1	高等数学	无
C2	程序设计	无
C3	离散数学	C1
C4	数据结构	C2, C3
C5	编译原理	C2, C4

C6	操作系统	C4, C7
C7	计算机组成原理	C2

课程依赖关系图：C1 → C3 → C4, C2 → C4, C2 → C7, C4 → C5, C4 → C6, C7 → C6

算法步骤：

1. 从有向图中选择一个没有前驱（即入度为 0）的顶点并且输出它
2. 从图中删去该顶点，并且删去从该顶点发出的全部有向边
3. 重复上述两步，直到剩余的图中不再存在没有前驱的顶点为止

如果还有顶点却没有入度为 0 的顶点，说明有向图有环存在。

拓扑排序的实现

数据结构详解

为每个顶点设立一个链表，每个链表有一个表头节点，这些表头节点构成一个数组，表头节点中增加一个存放顶点入度的域 count：

```
typedef struct /* 表头节点类型 */
{
    Vertex data;          /* 顶点信息 */
    int count;            /* 存放顶点入度 */
    ArcNode *firstarc;    /* 指向第一条弧 */
} VNode;
```

关键概念解释：

- **弧 (Arc)**：就是有向边。在有向图中，边是有方向的，所以称为“弧”。
- **第一条弧**：firstarc 是一个指针，指向从该顶点出发的**第一条有向边**。

邻接表结构图解：

假设有一个图：顶点 0 指向顶点 1 和 2，顶点 1 指向顶点 3，顶点 2 指向顶点 3。

邻接表结构如下：

```
adj[0]: [count=0, firstarc] → [adjvex=1, nextarc] → [adjvex=2, nextarc=NULL]
           ↑               ↑               ↑
        顶点0的入度      第一条弧0→1      第二条弧0→2

adj[1]: [count=1, firstarc] → [adjvex=3, nextarc=NULL]
           ↑               ↑
        顶点1的入度      弧1→3

adj[2]: [count=1, firstarc] → [adjvex=3, nextarc=NULL]
           ↑               ↑
        顶点2的入度      弧2→3

adj[3]: [count=2, firstarc=NULL]
           ↑
        顶点3的入度（没有出边）
```

所以 adj[i].firstarc 指向的就是从顶点 i 出发的第一条边！

拓扑排序算法代码详解：

```
void TopSort(VNode adj[], int n)
{
    int i, j;
    int St[MAXV], top = -1; /* 栈St的指针为top */
```

```

ArcNode *p;

/* ===== 第一步：把所有入度为0的顶点压入栈 ===== */
for (i = 0; i < n; i++)
    if (adj[i].count == 0) /* 如果顶点i的入度为0 */
    {
        top++;
        St[top] = i;      /* 把i压入栈 */
    }

/* ===== 第二步：不断从栈中取出顶点处理 ===== */
while (top > -1) /* 栈不为空时循环 */
{
    i = St[top];
    top--; /* 出栈：取出一个入度为0的顶点i */
    printf("%d ", i); /* 输出顶点i（拓扑序的一部分） */

    /* ===== 第三步：遍历顶点i的所有出边 ===== */
    p = adj[i].firstarc; /* p指向从i出发的第一条弧 */
    while (p != NULL) /* 只要还有出边就继续 */
    {
        j = p->adjvex; /* j是这条弧指向的顶点 */
        adj[j].count--; /* j的入度减1（因为i已经处理了） */

        /* 如果j的入度变成0了，把j压入栈 */
        if (adj[j].count == 0)
        {
            top++;
            St[top] = j;
        }

        p = p->nextarc; /* 找下一条从i出发的弧 */
    }
}
}

```

算法执行过程示例：

以上面的图为例：

1. **初始状态**：栈中有 [0]（只有顶点 0 入度为 0）
2. **取出 0**，输出“0”。处理边 0→1 和 0→2：
 - 顶点 1 的入度从 1 变为 0，压入栈
 - 顶点 2 的入度从 1 变为 0，压入栈
 - 现在栈中有 [1, 2]
3. **取出 2**，输出“0 2”。处理边 2→3：
 - 顶点 3 的入度从 2 变为 1
 - 栈中还有 [1]
4. **取出 1**，输出“0 2 1”。处理边 1→3：
 - 顶点 3 的入度从 1 变为 0，压入栈
 - 栈中有 [3]
5. **取出 3**，输出“0 2 1 3”。没有出边。
6. **栈空，结束**

拓扑序：0 → 2 → 1 → 3

拓扑排序性质总结：

- 拓扑排序只能应用于有向无环图
- 对于一个给定的图，拓扑序可能不唯一

- 如果序列 X 是图 G 的拓扑序，则“序列 X 的逆序”是“ G 的转置图”的拓扑序

关键路径基本概念

在 PT 图 (Potential task graph) 中：

- 节点表示工序
- 有向边 (i, j) 表示工序 i 完成之后工序 j 才能启动
- 边权 w_i 表示工序 i 所需的时间

在 AOE 网 (Activity On Edge network) 中：

- **关键路径**：从源点到汇点的最长路径
- **关键活动**：关键路径上的活动
- 关键路径决定了整个工程的最短完成时间

PERT 图简介 (选学)

PERT 图 (Programme Evaluation and Review Technique) :

- 节点表示工序之间的关系 (事件)
- v_k 是 e_i 的终点、 e_j 的始点，表示工序 e_i 完成后 e_j 才能开始
- 有向边表示工序
- 边权 w_i 表示该工序所需时间

PERT 图与 PT 图的比较：

- PERT 的节点和边数更少些，省内存
- PT 图的节点数等于工序数，更加固定，易于编程
- PT 图更加灵活，能适应一些额外的约束

最早启动时间

当节点重新编号以后，设 v_1 到各点最长路径长度分别是 $0 = \pi(1'), \pi(2'), \dots, \pi(n')$ ，则：

$$\pi(j') = \max_{0 < i' < j'} (\pi(i') + l_{i'j'})$$

其中 i' 是 j' 的前驱， $\pi(i')$ 就是工序 i' 的最早启动时间。

关键路径算法：

1. 根据定理 2.7.1 对节点重新编号为 $v'_1, v'_2, \dots, v'_n$
2. 赋初值 $\pi(v'_1) = 0$
3. 依次更新 $\pi(v'_j), j = 2, 3, \dots, n$

$$\pi(v'_j) = \max_{v'_i \in \Gamma^-(v'_j)} (\pi(v'_i) + w(v'_i, v'_j))$$

4. 结束

算法复杂性： $O(m)$

最晚启动时间与最大允许延误时间

最晚启动时间：对于拓扑序列 $v'_1, v'_2, \dots, v'_n$ ， v'_i 的最晚启动时间依据的是为 v'_i 到 v'_n 的最长路径的长度。

最大允许延误时间算法：

1. 根据定理 2.7.1 对节点重新编号为 $v'_1, v'_2, \dots, v'_n$
2. 赋初值 $\tau(v'_n) = p(v'_n)$
3. 依次更新 $\tau(v'_j), j = n - 1, \dots, 1$

$$\tau(v'_j) = \min_{v'_i \in \Gamma^+(v'_j)} (\tau(v'_i) - w(v'_j, v'_i))$$

4. 结束

最早启动求“最大值”，最晚启动求“最小值”。

各工序允许延误时间 $t(v'_j)$ 等于各工序最晚启动时间减去各工序最早启动时间：

$$t(v'_j) = \tau(v'_j) - \pi(v'_j)$$

(8) 中国邮路

问题提出

我国管梅谷教授在 1960 年首先提出：邮递员从邮局出发，走遍辖区内所有的街道至少一次（可以重复），最后回到邮局。要走怎样的路线全程才最短？

图论模型： 在一个正权连通图 G 中，从某点出发经过每条边至少一次最后返回出发点的最短回路称为**最佳邮路（中国邮路）**。这是一个既与 Euler 图有关又与最短路有关的问题。

无向图的中国邮路

非欧拉图 G 的任何最佳邮路，必然通过某些边将超过一次。中国邮递员问题就变化为：给出一个连通图 G ：

1. 求 E_1 是 E 的子集满足条件：在 G 中重复 E_1 中每条边，使得到的图 G^* 是 Euler 图（称这样的 E_1 为可行集）。并且使其边权和尽可能小（称这样的可行集 E_1 为最优集）。
2. 求 G^* 的 Euler 回路。

回顾：

- 当 G 所有节点度都是偶数时，即 G 有欧拉回路，该回路就是最佳邮路
- 当 G 只有 2 个节点的度为奇时，存在一条欧拉道路，该道路加上从起点到终点的最短路径组成的回路就是最佳邮路
- 当奇数度节点的个数大于 2 时？

定理 2.8.1

定理 2.8.1: L 是无向连通图 G 最佳邮路的充要条件是：

1. G 的每条边最多重复一次。
2. 在 G 的任意一个回路上，重复边的长度之和不超过该回路长度的一半。

证明：

必要性 (1. 最佳邮路 G 中每条边最多重复一次)：

- 若一条最佳邮路重复经过图的某些边，将 G 中 k 次重复的边画 k 次，得到 G' 。
- 设最佳邮路 L' 使 G 中边 e_{ij} 重复 $n(n > 1)$ 次，这时 G' 中有欧拉回路 L' ，若使 e_{ij} 重复 $n - 2$ 次，得到 G'' ， G'' 各点度仍是偶数
- G'' 的欧拉回路 L'' 也是 G 的一条中国邮路，且 L'' 长度小于 L' ，与 L' 是最佳邮路矛盾。因此边 e_{ij} 最多重复一次。

必要性 (2. 任一回路重复边长不超过回路长度一半)：

- 假设 G 中某个回路 C 的重复边的长度大于 C 总长度的一半
- 令 C 中重复的边不重复，不重复的边重复，得到 G''
- G'' 仍是欧拉图，因为回路上每个顶点的度数改变 0 或 2，不会改变欧拉图的性质。且 L'' 长度小于 L' ，与 L' 是最佳邮路矛盾
- 因此，在任意一个回路上，重复边的长度之和不会超过回路的一半

充分性 (无向图满足这两个条件后成为的欧拉图必为最佳邮路)： 只需证明凡满足定理中条件(1)和(2)的重复边集，其总长度均相等即可。

- 设 $L_1 = E(G) + Q + Q_1$, $L_2 = E(G) + Q + Q_2$ ，其中 Q 是 L_1 和 L_2 共同的重复边集， Q_1 和 Q_2 是分别只属于 L_1 和 L_2 的重复边集， L_1 和 L_2 的节点度数均为偶数
- 设对称差 $E'(G) = Q_1 + Q_2$ ， $G' = (V(G), E'(G))$ 为简单图，可能不连通，但其每个节点的度数均为偶数（因为 $L_1 + L_2$ 为偶数，减去 $(2E(G) + 2Q)$ ，每个节点度数都减去 2 的倍数，仍为偶数），所以图 G' 可分为若干个回路，所有节点度数均为偶数。
- 故可构造 $G' = (V(G), E'(G))$ ， G' 是简单图，各节点度是偶数
- 若 $E'(G) = \Phi$ ，显然 $\pi(L_1) = \pi(L_2)$
- 否则（即 $E'(G) \neq \Phi$ ，其中对称差 $E'(G) = Q_1 + Q_2$ ）因为 G' 是简单图，各节点度是偶数，可分为若干个回路。对任一个回路 C ，设 C_1 和 C_2 分别是 L_1 和 L_2 的重复边集。由已知条件(2)可知， $\pi(C_1) \leq \pi(C_2)$ ， $\pi(C_2) \leq \pi(C_1)$ 。因此 $\pi(C_1) = \pi(C_2)$ ，故 $\pi(L_1) = \pi(L_2)$ 。充分性得证。

定理给出了中国邮路问题的一种算法称为“奇偶点图上作业法”。

奇偶点图上作业法

构造中国邮路算法：

1. 找出度为奇的点
2. 依据条件 1 构造邮路，即 G 的每条边最多重复一次，并保证计算重复边之后度都是偶数
3. 由条件 2 对所有回路进行判断，在 G 的任意一个回路上，如果重复边的长度之和超过该回路长度的一半，则令回路中的重复边不重复，不重复边变为重复

Edmonds 最小权匹配算法

Edmonds 最小权匹配算法 (1973 年) :

1. 确定 G 中度为奇的节点，构成 $V_0(G)$
2. 求 $V_0(G)$ 各节点在 G 中的最短路径 P_{ij} 及其长度 $\pi(v_i, v_j)$
3. 对 $V_0(G)$ 的节点进行最小权匹配，即选出 $|V_0(G)|/2$ 个 $\pi(v_i, v_j)$ ，保证每个节点在 P_{ij} 中只出现一次，并且这些 $\pi(v_i, v_j)$ 的总和最小（最佳匹配问题，在后面二分图匹配中会讲如何实现）
4. 在最小权匹配里各 $\pi(v_i, v_j)$ 所对应的路径 P_{ij} 中的各边在 G 中重复一次，得到 G'
5. G' 是欧拉图，它的一条欧拉回路即为解

有向图的中国邮路

有向（连通）图可能没有中国邮路：

- 当图中含有正度或负度为 0 的点时，没有中国邮路
- 若图中各点正负度相同，则存在有向欧拉回路，任一条欧拉回路都是问题的解
- 对于一般的非对称有向图（即存在正负度不相等的节点），如何确定中国邮路？

存在正负度不相等的节点时：

- $d'(v_i) = d^-(v_i) - d^+(v_i)$
- 若 $d'(v_i) > 0$ ，表示需要 $d'(v_i)$ 次重复经过 v_i 发出的一些边，或者说 v_i 可供应 $d'(v_i)$ 个单位量。
- 若 $d'(v_i) < 0$ ，表示需要 $d'(v_i)$ 次重复经过进入 v_i 的一些边，或者说 v_i 可接收 $d'(v_i)$ 个单位量。
- 若 $d'(v_i) = 0$ ，称 v_i 是中间节点。

构造有向图的中国邮路算法：

1. 计算各点的正负度，求出 $d'(v_i) = d^-(v_i) - d^+(v_i)$
2. 添加一个超发点 v_s ，对满足 $d'(v_i) > 0$ 的节点，加入 $d'(v_i)$ 条有向边 (v_s, v_i) ，权为 0
3. 添加一个超收点 v_t ，对满足 $d'(v_j) < 0$ 的节点，加入 $d'(v_j)$ 条有向边 (v_j, v_t) ，权为 0。得到图 G' 。
4. 在 G' 中求 $d(v_s)$ 条过以 v_s, v_t 为两端点的形如 $(v_s, v_i), (v_j, v_t)$ ，每边一次且仅一次的总和最小的 P - s - t 道路。记下 G 中各边在这些道路里的重复次数。
5. 计入各边的重复次数， G 中存在有向欧拉回路，其中一条即为解。

💧 图论：血泪教训与阴间套路总结 💧

记录于：2026-03-18 | 耗时：5 小时的心态爆炸

1. 核心铁律：只要边数够多，必有妖孽

在简单图中， n 个点如果想要“安分守己”（连通且无环），最多只能有 $n - 1$ 条边。

- **定理：** 若 $m \geq n$ ，则 **图中必有回路 (Cycle)**。
- **技巧：** 证明此结论最快的两把刀：
 1. **数学归纳法：** 找到度数 $d(v) \leq 1$ 的废点，删掉它，把锅甩给 $n - 1$ 的情况。
 2. **最长路径法：** 在全员度数 $d \geq 2$ 时，拉出一条最长路径，终点因为没处可去，只能**往回连老熟人**，瞬间闭环。

2. 情况 B 的必杀技：最长路径论证 (Longest Path Argument)

这是今天卡了最久的坑。当图里所有点都“很顶”（度数 $d \geq 3$ 或 $d \geq 2$ ）时，不要空想，直接祭出此招：

- **严谨步骤：**
 1. 设 $P = (v_1, v_2, \dots, v_k)$ 为图中最长路径。
 2. 终点 v_k 的所有邻居 $N(v_k)$ **必须全部在路径 P 上**（否则路径能更长，矛盾！）。
 3. 如果 $d(v_k) \geq 2$ ，它至少连了一个之前的点 v_i ，形成**回路**。
 4. 如果 $d(v_k) \geq 3$ ，它至少连了两个之前的点 v_i, v_j ，在大圈上又拉了一根捷径，形成**带弦回路 (Chorded Cycle)**。

3. 无三角形 (Triangle-free) 的物理限制

看到“无三角形”，脑子里第一反应不是组合数，而是**朋友圈互斥**。

• **核心推导：**

- G 无三角形 $\Rightarrow$ 任何相邻两点 u, v , 其邻居集合 **交集为空**。
- 导出重要不等式: $d(u) + d(v) \leq n$ 。
- 所有的平方和证明 ($\sum d^2$) 和边数上限证明 ($m \leq \frac{n^2}{4}$), 全部是从这一个 **物理隔绝** 的点出发的。

4. 考试/自救心态建设

- **教训：** 图论证明题不是靠逻辑硬掰的, 是靠 **辅助结构** (最长路径、极大图、二部图) 切入的。
- **反思：** 以后死磕超过 30 分钟没思路, 立刻检查是否漏掉了以下套路:
 - **删点减规模** (归纳法)
 - **最长路径找端点** (构造回路)
 - **二部图缩放** (极值边界)

“五个小时不会白费, 这些套路现在是我的了。”

五、树 2026-04-09

(1) 树的定义

背景： 英国数学家 Arthur Cayley (1821-1895) 最早提出树的概念 (1857 年), 在研究 $C_n H_{2n+2}$ 的化合物的同分异构体时提出。树在计算机科学中有非常重要的作用: 目录树、搜索树、判定树、分类树、语法树、编码树等。

定义 3.1.1: 一个不含任何回路的连通图称为**树**, 用 T 表示一棵树。若图不含任何回路, 则称为**林**。

- T 中的边称为**树枝**
- 度为 1 的节点称为**树叶**
- 度大于 1 的节点为**分支节点** (内节点)

(2) 树的等价定义

定理 3.1.2: 设 $G = \langle V, E \rangle$ 是 n 阶 m 条边的无向图, 则下面各命题是等价的, 都是树的定义:

1. G 是树 (连通且无回路)
2. G 中任意两个顶点之间存在唯一的路径
3. G 中无回路, 且 $m = n - 1$
4. G 是连通的, 且 $m = n - 1$
5. G 是连通的, 且 G 中任何边均为桥
6. G 中没有回路, 但在任何两个不同的顶点之间加一条新边, 在所得图中得到唯一一个含新边的回路

证明

(1) $\Rightarrow$ (2): 由 G 的连通性可知 $\forall u, v \in V$, u 与 v 之间存在路径。若路径不是唯一的, 设 Γ_1 与 Γ_2 都是 u 到 v 的路径, 则必存在由 Γ_1 和 Γ_2 上边构成的回路, 与 G 中无回路矛盾。

(2) $\Rightarrow$ (3): 先证明 G 中无回路。若存在环或长度 ≥ 2 的圈, 则存在两条不同路径, 矛盾。再用归纳法证明 $m = n - 1$: $n = 1$ 显然; 设 $n \leq k$ 时成立; 当 $n = k + 1$ 时, 设 $e = (u, v)$ 为 G 中一条边, $G - e$ 有两个连通分支 G_1, G_2 , 由归纳假设 $m_i = n_i - 1$, 于是 $m = m_1 + m_2 + 1 = n_1 + n_2 - 2 + 1 = n - 1$ 。

(3) $\Rightarrow$ (4): 反证法。设 G 由 $s (s \geq 2)$ 个连通分支 $G_1, G_2, \dots, G_s$ 组成, 各连通分支均为树, 由 (1) $\Rightarrow$ (2) $\Rightarrow$ (3) 可知 $m_i = n_i - 1$, 于是 $m = \sum n_i - s = n - s$ 。由于 $s \geq 2$, 与 $m = n - 1$ 矛盾。

(4) $\Rightarrow$ (5): 需证明 G 中每条边均为桥。 $\forall e \in E, |E(G - e)| = n - 1 - 1 = n - 2$ 。 n 阶 m 条边的无向连通图 $m \geq n - 1$, 故 $G - e$ 不是连通图, e 为桥。

(5) $\Rightarrow$ (6): 由于 G 中每条边均为桥, 因此 G 中无圈。又由于 G 连通, 所以 G 为树。由 (1) $\Rightarrow$ (2) 可知 $\forall u, v \in V$ 且 $u \neq v$, u 与 v 之间存在唯一的路径 Γ , 则 $\Gamma \cup (u, v)$ 为 G 中的回路, 显然该回路是唯一的。

(6) $\Rightarrow$ (1): 需证 G 是连通的。 $\forall u, v \in V$ 且 $u \neq v$, 则 $(u, v) \cup G$ 产生唯一的回路, 有 $C - (u, v)$ 为 G 中 u 到 v 的通路, 故 u 可达 v 。由 u 和 v 的任意性可知 G 是连通的。

例 3.1.2: 7 阶无向树有 3 个叶节点和 1 个 3 度顶点, 其余 3 个节点的度数均不是 1 和 3。试画出满足要求的所有非同构的无向树。

解: 设 T_i 为满足要求的无向树, 则边数 $m = 6$ 。不妨假设 v_1, v_2, v_3 是叶节点, v_7 是 3 度节点, v_4, v_5, v_6 是待定节点。于是 $\sum d(v_j) = 12 = 3 \times 1 + 3 + d(v_4) + d(v_5) + d(v_6)$ 。由于 $d(v_j) \neq 1$ 且 $d(v_j) \neq 3$, 可知 $d(v_j) =$

$2(j = 4, 5, 6)$ 。度数列 $1, 1, 1, 2, 2, 2, 3$ 。 v_7 的邻域 $N(v_7)$ 中 3 个顶点的度数列只能是 $(1, 1, 2), (1, 2, 2), (2, 2, 2)$ 三种，产生三棵非同构的 7 阶无向树。

(3) 支撑树

定义 3.1.3: 设树 T 是无向图 G 的子图，则称 T 为 G 的树；若树 T 是 G 的支撑子图，则称 T 是 G 的**支撑树 (Spanning Tree)**。

- 设 T 是 G 的支撑树， $\forall e \in E(G)$ ，若 $e \in E(T)$ ，则称 e 为 T 的**树枝**，否则称 e 为 T 的**弦**
- 称子图 $G[E(G) - E(T)]$ 为 G 关于 T 的**余树**
- 余树不一定连通，但含回路

生成支撑树的方法

破圈法:

- 每次去掉回路中的一条边
- 去掉边的总数为 $m - n + 1$
- 计算效率低

避圈法:

- 每次选取 G 中一条与已选取的边不构成回路的边
- 增加的边的总数为 $n - 1$
- 一般用深度优先或广度优先搜索来实现

深度优先搜索 (DFS):

- 任选图中的一个节点作根
- 通过相继地增加边来形成从这个顶点开始的通路，每条新边都和通路上的最后一个节点以及还不在通路上的节点关联
- 若通路经过所有节点，则为支撑树；否则从通路上倒退一个节点，形成该节点开始的尚未访问过的节点的通路
- 重复直到不能增加新边

广度优先搜索 (BFS):

- 任选图中的一个节点作根
- 添加与这个节点相关联的所有边，形成支撑树中第一层的所有节点
- 按顺序访问一层上的每个节点，只要不产生简单回路，就将与这个节点关联的每条边添加到树中
- 重复直到已经添加了图中所有的节点

生成所有支撑树的方法

问题：如何枚举一个图的所有支撑树？

方法一：基本回路法

- 任取一棵支撑树 T
- 对于每条弦 e ， $T + e$ 产生唯一的回路（基本回路）
- 枚举每条弦对应的基本回路，每条弦可替换回路中的一条树枝
- 时间复杂度： $O(\tau(G) \times m)$ ，其中 $\tau(G)$ 为支撑树数目

方法二：删边/收缩法（递归）

- 设 $\tau(G)$ 为 G 的支撑树数目
- 对任意边 e ： $\tau(G) = \tau(G - e) + \tau(G/e)$
 - $\tau(G - e)$ ：不含 e 的支撑树数目（删除边 e ）
 - $\tau(G/e)$ ：必含 e 的支撑树数目（收缩边 e ）
- 递归分解，直到图变成树或不再连通

方法三：Gabow-Myers 算法

- 利用关联矩阵的性质
- 通过矩阵行列式计算定位所有支撑树
- 适用于稠密图，时间复杂度 $O(\tau(G) \times n + m)$

方法四：Read-Tarjan 算法

- 经典的支撑树枚举算法
- 时间复杂度 $O(\tau(G) + n + m)$ ，是最优的

- 使用回溯策略，避免重复计算

最小生成树

问题：如何找到边权和最小的支撑树？

定义： 设连通加权图 $G = \langle V, E, w \rangle$ ，其中 $w: E \rightarrow R^+$ 为边权函数。称 T 为 G 的**最小生成树 (Minimum Spanning Tree, MST)**，若 T 是 G 的支撑树且 $w(T) = \sum_{e \in T} w(e)$ 最小。

Kruskal 算法（避圈法的加权版本）：

1. 将所有边按权值从小到大排序
2. 依次考察每条边，若该边与已选边不构成回路，则加入
3. 直到选取了 $n - 1$ 条边

正确性： 贪心选择性质 + 切割定理

时间复杂度： $O(m \log m)$ （排序）或 $O(m\alpha(n))$ （用 Fibonacci 堆）

Prim 算法：

1. 任选一个顶点作为初始树
- 每次选择与当前树相邻的权值最小的边
- 将该边及其不在树中的端点加入树
- 重复直到树包含所有顶点

正确性： 切割定理保证

时间复杂度：

- 使用邻接矩阵： $O(n^2)$
- 使用二叉堆： $O(m \log n)$
- 使用 Fibonacci 堆： $O(m + n \log n)$

切割定理： 设 S 是 V 的任意非空真子集，若 e 是跨越 S 与 $V - S$ 的最小权边，则存在包含 e 的最小生成树。

唯一性：

- 若所有边权值互不相同，则 MST 唯一
- 若存在相同权值的边，MST 可能不唯一，但所有 MST 的权值相同

应用： 网络设计、电路布线、聚类分析、图像分割

(4) 基本关联矩阵及其性质

线性相关预备知识

设 V 是数域 K 上的向量空间，如果对 V 中的 n 个向量 $v_1, v_2, \dots, v_n$ ，在数域 K 中存在不全为零的 n 个数 $a_1, a_2, \dots, a_n$ ，使得 $a_1 v_1 + a_2 v_2 + \dots + a_n v_n = 0$ ，则称 $v_1, v_2, \dots, v_n$ **线性相关**；反之则称**线性无关**。

性质：

1. 若一向量组线性无关，即使每一向量都在相同位置处增加一分量，仍然线性无关
2. 若一向量组线性相关，即使每一向量都在相同位置处减去一分量，仍然线性相关
3. 若 $v_1, v_2, \dots, v_n$ 线性无关，而 $u, v_1, \dots, v_n$ 线性相关，则 u 必可由 $v_1, \dots, v_n$ 线性表示，且表示方法唯一
4. 若一向量组线性相关，则补充若干向量后依然线性相关
5. 若一向量组线性无关，则其任意子集组成的向量组依然线性无关

秩： 矩阵 A 的列秩是 A 的线性无关列向量组的极大向量数；矩阵的行秩等于其列秩。初等变换不影响矩阵的行秩和列秩。将矩阵 A 的秩记作 $r(A)$ 。

行列式： 将一个 $n \times n$ 的矩阵 A 映射到一个标量，记作 $\det(A)$ 或 $|A|$ 。

余子式： 在行列式 $D = |a_{ij}|_{n \times n}$ 中，去掉第 i 行与第 j 列全部元素后所得的 $(n - 1)$ 阶行列式，称为元素 a_{ij} 的**余子式** M_{ij} 。 $A_{ij} = (-1)^{i+j} M_{ij}$ 称为**代数余子式**。

关联矩阵

无环有向图的关联矩阵： 设无环有向图 $D = \langle V, E \rangle$ ， $V = \{v_1, v_2, \dots, v_n\}$ ， $E = \{e_1, e_2, \dots, e_m\}$ ，令

$$m_{ij} = \begin{cases} 1, & v_i \text{ 是边 } e_j \text{ 的始点} \\ 0, & v_i \text{ 与边 } e_j \text{ 不关联} \\ -1, & v_i \text{ 是边 } e_j \text{ 的终点} \end{cases}$$

则称 $(m_{ij})_{n \times m}$ 为 D 的关联矩阵, 记为 $M(D)$ 。

性质:

1. $\sum_{i=1}^n m_{ij} = 0, j = 1, 2, \dots, m$
2. 第 i 行 1 的个数等于 $d^+(v_i)$, -1 的个数等于 $d^-(v_i)$
3. e_j 与 e_k 是重边 $\leftrightarrow$ 第 j 列与第 k 列相同

例: 下图有 6 个顶点 9 条边, 其关联矩阵 $B(G)$ 为 6×9 矩阵:

$$B(G) = \begin{pmatrix} 1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & -1 & 1 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & -1 & -1 \\ 0 & 1 & -1 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

基本关联矩阵 B_3 (划去 v_3 对应的行), 为 5×9 矩阵:

$$B_3 = \begin{pmatrix} 1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & -1 & 1 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & -1 & -1 \\ 0 & 1 & -1 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

关联矩阵的秩

定理 3.2.1: 有向图 $G = (V, E)$ 关联矩阵 B 的秩 $\text{ran } B < n$ 。

证明: B 中每列都只有 1 和 -1 两个非零元素, 因此 B 的任意 $n-1$ 行加到第 n 行后, 第 n 行全为 0, 即 B 的 n 个行向量线性相关。

引理: 设 B 为有向连通图 G 的关联矩阵, C 是 G 中的一个回路, 则 C 中各边所对应 B 的各列线性相关。

证明: 设 C 包含 l 个点, 这 l 条边对应关联矩阵 B 的 l 列, 构成子阵 $B(G_C)$ 。由于 C 连通, 由 C 构成的关联矩阵 $B(C)$ 是 l 阶方阵, $\text{ran}(B(C)) = l-1$, 即 $B(C)$ 的 l 列线性相关。由于 $B(G_C)$ 中其余节点所对应的行元素全为零, 所以 $B(G_C)$ 的 l 列仍线性相关。

定理 3.2.3: 有向连通图 $G = (V, E)$ 关联矩阵 B 的秩 $\text{ran}(B) = n-1$ 。

证明: 由定理 3.2.1, $\text{ran } B \leq n-1$, 故只需证 $\text{ran } B \geq n-1$ 。设 B 中线性相关最少的行数为 l , 设这 l 行对应点 $v_{i_1}, v_{i_2}, \dots, v_{i_l}$ 。由于 B 每列只有 2 个非零元, 所以这 l 个行向量中每列最多只有 2 个非零元。对 B 做行列交换, 将含有 2 个非零元的列移到前 l 列, 其余列全为 0。若 $n-l > 0$, 则 G 至少分为 2 个连通分支, 与 G 连通矛盾。故 $l = n$, B 中最少需要 n 行才能线性相关, 而任何 $n-1$ 行线性无关, $\text{ran}(B) = n-1$ 。

基本关联矩阵

基本关联矩阵: 在 $G = \langle V, E \rangle$ 的关联矩阵 B 中划去任意点 v_k 所对应的行, 得到一个 $(n-1) \times m$ 矩阵 B_k 。

定理 3.2.4: 有向连通图 G 的基本关联矩阵 B_k 的秩为 $n-1$ 。

推论 3.2.1: n 个点的树的基本关联矩阵的秩为 $n-1$ 。

定理 3.2.5: 设 B_k 为有向连通图 G 的基本关联矩阵, C 是 G 中的一个回路, 则 C 中各边所对应 B_k 的各列线性相关。

证明: 由引理可知 $B(G_C)$ 的 l 列线性相关。由线性相关的性质 (若一向量组线性相关, 即使每一向量都在相同位置处减去一分量, 仍然线性相关), 因此 $B_k(G_C)$ 的各列也线性相关。

推论 3.2.2: 设 H 是有向连通图 G 的子图。若 H 含有回路, 则 H 的诸边对应的 G 的基本关联矩阵各列线性相关。

定理 3.2.6: 令 B_k 是有向连通图 G 的基本关联矩阵, 则 B_k 的任意 $n-1$ 阶子阵行列式 $\neq 0 \leftrightarrow$ 其各列所对应的边构成 G 的一棵支撑树。

证明 (必要性): 若某个 $n-1$ 阶子阵 $B_{k(G_T)}$ 的行列式非零, 由推论 3.2.2, T 中不含回路。因为 $B_{k(G_T)}$ 是 $(n-1)$ 阶子阵, 所以其包含 n 个节点、 $n-1$ 条边, 根据树的等价定义(4), T 是 G 的一棵支撑树。

证明 (充分性): 设 T 是 G 的一棵树, 包含 n 个节点、 $n-1$ 条边。子图 T 的基本关联矩阵 $B_{k(T)}$ 是 $(n-1)$ 阶方阵, 其秩 $\text{ran } B_{k(T)} = n-1$, 所以行列式非零。它恰好对应 B_k 的某个 $n-1$ 阶子阵。

(5) 支撑树的计数

问题: 一个图有多少棵不同的支撑树?

答案: 由**矩阵树定理 (Matrix-Tree Theorem)** 计算。设 B_k 是有向连通图 G 的某一基本关联矩阵, 则 G 的不同支撑树数目为 $\tau(G) = \det(B_k B_k^T)$ 。

特例:

- 完全图 K_n 的支撑树数目为 n^{n-2} (Cayley 公式)
- 完全二部图 $K_{p,q}$ 的支撑树数目为 $p^{q-1}q^{p-1}$

Binet-Cauchy 定理

定理 3.3.1 (Binet-Cauchy 定理): $A = (a_{ij})_{m \times n}$, $B = (b_{ij})_{n \times m}$, $m \leq n$, 则

$$\det(AB) = \sum_i A_i B_i$$

其中 A_i 是从 A 中取不同的 m 列所成的行列式, B_i 是从 B 中取相应的 m 行构成的行列式。

注意: 该定理揭示了乘积矩阵的行列式与各矩阵子行列式之间的关系, 连通图 G 不同支撑树的计数恰好利用了这种关系。

定理 3.2.2: 设 B_0 为 $B(G)$ 的任意 k 阶方阵, 则 $|B_0| = \pm 1$ 或 0 。

证明: 对 k 归纳。 $k=1$ 时成立。假设 $k-1$ 时成立, 因为 B_0 为 B 的子阵, 每列最多只有 2 个非零元。若某一列全为 0 或每列恰好有 2 个非零元, 则 $|B_0| = 0$ 。假设存在只有一个非零元的列, 则按该列展开后用归纳法即可。

支撑树计数公式

定理 3.3.2: 设 B_k 是有向连通图 $G = \langle V, E \rangle$ 的某一基本关联矩阵, 则 G 的不同树的数目是 $\det(B_k B_k^T)$ 。

证明: 设 $B_k = (b_{ij})_{(n-1) \times m}$ 。由 Binet-Cauchy 定理:

$$\det(B_k B_k^T) = \sum_i |B_i| |B_i| = \sum_i |B_i|^2$$

其中 $|B_i|$ 为 B_k 的某一 $n-1$ 阶子阵的行列式。若 $|B_i|^2 \neq 0$, 则 $|B_i| \neq 0$, 其所对应的边构成 G 的一棵树。因为 $|B_i| = \pm 1$, 所以如果 B_i 各列所对应的边构成 G 的一棵树, 则对 $\det(B_k B_k^T)$ 的贡献为 1, 恰为 G 中不同树的数目。

例: 求右图支撑树的数目。(4 个顶点 v_1, v_2, v_3, v_4 , 5 条边)

解: 关联矩阵

$$B(G) = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 \\ 0 & -1 & -1 & 1 & 0 \\ 0 & 0 & 0 & -1 & 1 \\ -1 & 0 & 1 & 0 & -1 \end{pmatrix}$$

取基本关联矩阵 B_3 (划去 v_3 对应的行):

$$B_3 = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 \\ 0 & -1 & -1 & 1 & 0 \\ -1 & 0 & 1 & 0 & -1 \end{pmatrix}$$

$$B_3 B_3^T = \begin{pmatrix} 2 & -1 & -1 \\ -1 & 3 & -1 \\ -1 & -1 & 3 \end{pmatrix}$$

$$\det(B_3 B_3^T) = 18 - 1 - 1 - 3 - 2 - 3 = 8$$

特殊情况计数

- **不含 e 的支撑树数目**: 令 $G' = G - e$, 只需求 G' 的支撑树数目
- **必含 e 的支撑树数目**: G 的树数目减去 $G' = G - e$ 的树数目; 或将 e 的两个端点收缩成一个点, G' 的树与 G 的必含 e 的树一一对应

例: 求不含 e_1 的支撑树数目。

$$B(G') = \begin{pmatrix} 1 & 0 & 0 & 0 \\ -1 & -1 & 1 & 0 \\ 0 & 0 & -1 & 1 \\ 0 & 1 & 0 & -1 \end{pmatrix}$$

, 取 B_4 :

$$B_4 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ -1 & -1 & 1 & 0 \\ 0 & 0 & -1 & 1 \end{pmatrix}$$

,

$$B_4 B_4^T = \begin{pmatrix} 1 & -1 & 0 \\ -1 & 3 & -1 \\ 0 & -1 & 2 \end{pmatrix}$$

$$\det(B_4 B_4^T) = 6 - 1 - 2 = 3$$

例: 求含 e_1 的支撑树数目 (收缩法)。将 e_1 的两端点 v_1, v_2 收缩为一个点, 得到新图 G' 。

$$B_{v_1 v_2} = \begin{pmatrix} 0 & 0 & -1 & 1 \\ -1 & 1 & 0 & -1 \end{pmatrix}$$

,

$$B_{v_1 v_2} B_{v_1 v_2}^T = \begin{pmatrix} 2 & -1 \\ -1 & 3 \end{pmatrix}$$

$$\det = 6 - 1 = 5$$

例: 求完全图 K_n 中不同支撑树的数目。

$$B_k^T B_k = \begin{pmatrix} n-1 & -1 & \dots & -1 \\ -1 & n-1 & \dots & -1 \\ \dots & \dots & \dots & \dots \\ -1 & -1 & \dots & n-1 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 1 & \dots & 1 \\ 0 & n & \dots & 0 \\ \dots & \dots & \dots & \dots \\ 0 & 0 & \dots & n \end{pmatrix}$$

$$\det(B_k B_k^T) = n^{n-2}$$

无向图的处理方法: 将无向图 G 的各边加一方向, 得定向图 G' , G' 的树与 G 的树一一对应。

(6) 根树

定义 3.3.1: T 是有向树, 若 T 中存在某节点 v_0 的入度为 0, 其余节点入度为 1, 则称 T 是以 v_0 为根的**外向树**, 或称**根树**, 用 T 表示。

根树的特征: 若对根树的节点和边序号重新编号, 使得每条边 $e_j = (v_{i_j}, v_{j_j})$ 都满足 i_j 的编号小于 j 的编号, 则得到根节点基本关联矩阵 B'_0 为上三角矩阵, 下三角为 0, 对角元均为 -1 。若把该矩阵的所有 1 均变为 0, 行列式不变。而其他的树一定存在节点 (如根节点) 入度为 0, 修改后出现全 0 行, 行列式值变为 0。

根树计数

定理 3.3.3: 设 B_k 表示有向连通图 G 的基本关联矩阵, B_k^- 为 B_k 中将全部 1 改为 0 之后的矩阵, 则 G 中以 v_k 为根的根树数目是 $\det(B_k^-(B_k^-)^T)$ 。

证明:

- 由 Binet-Cauchy 定理, $\det(B_k^-(B_k^-)^T) = \sum_i B_i^- |B_i|$
- 若 $|B_i| \neq 0$, 说明这 $n-1$ 条边构成一棵树
- 此时 $B_i^- = |B_i|$, 因此它们在 $\det(B_k^-(B_k^-)^T)$ 中的贡献度为 1
- 由于遍历了所有 $n-1$ 条边的组合, 所以为 v_k 为根的根树数目

例: 求以 v_1 为根的根树数目。(4 个顶点 v_1, v_2, v_3, v_4 , 6 条边)

$$B(G) = \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 & -1 & 1 \\ -1 & 0 & 0 & -1 & 1 & 0 \end{pmatrix}$$

取 B_1 (划去 v_1 对应的行), 并将全部 1 改为 0 得到 B_1^- :

$$B_1 = \begin{pmatrix} 0 & 0 & -1 & 1 & 0 & -1 \\ 0 & -1 & 0 & 0 & -1 & 1 \\ -1 & 0 & 0 & -1 & 1 & 0 \end{pmatrix}$$

$$B_1^- = \begin{pmatrix} 0 & 0 & -1 & 0 & 0 & -1 \\ 0 & -1 & 0 & 0 & -1 & 0 \\ -1 & 0 & 0 & -1 & 0 & 0 \end{pmatrix}$$

$$B_1^-(B_1^-)^T = \begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix}$$

$$\det(B_1^-(B_1^-)^T) = 8 - 1 = 7$$

同理可求以 v_1 为根且不过 e_1 的根树数目: $\det(B_1^-(B_1^-)^T) = 4 - 1 = 3$

以 v_1 为根且必过 e_1 的根树数目: $7 - 3 = 4$

(7) 回路矩阵

重点: 回路矩阵的定义、性质、与关联矩阵的关系、计算方法

完全回路矩阵

定义 3.4.1 有向连通图 G 的全部初级回路构成的矩阵, 称为 G 的**完全回路矩阵**, 记为 C_e 。元素为:

$$c_{ij} = \begin{cases} 1, & e_j \in C_i \text{ 且与 } C_i \text{ 方向一致} \\ -1, & e_j \in C_i \text{ 且与 } C_i \text{ 方向相反} \\ 0, & \text{其他} \end{cases}$$

完全回路矩阵最多可能包含 $2^{m-n+1} - 1$ 个回路, 但不一定独立。

例: 求右图的完全回路矩阵。共 7 个回路 $C_1 C_7$:

回路矩阵

例：求右图的完全回路矩阵。

$$C_1 = v_1 v_2 v_3 v_1, C_2 = v_1 v_2 v_4 v_1, C_3 = v_1 v_2 v_3 v_4 v_1,$$

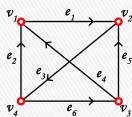
$$C_4 = v_1 v_2 v_4 v_3 v_1, C_5 = v_1 v_3 v_2 v_4 v_1, C_6 = v_1 v_3 v_4 v_1,$$

$$C_7 = v_2 v_3 v_4 v_2$$

G 的完全回路矩阵是：

$$C_e = \begin{matrix} C_1 & \begin{bmatrix} 1 & 0 & 0 & 1 & -1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & -1 & 0 & 0 & -1 & -1 \\ 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & -1 & 1 & 0 \\ 0 & 1 & 0 & -1 & 0 & -1 \\ 0 & 0 & 0 & -1 & 0 & -1 \end{bmatrix} \\ e_1 & e_2 & e_3 & e_4 & e_5 & e_6 \end{matrix}$$

$$C_1 \oplus C_2 = \{e_4, e_5, e_3, e_2\} = C_5$$



$$C_e = \begin{pmatrix} 1 & 0 & 0 & 1 & -1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & -1 & 0 & 0 & -1 & -1 \\ 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & -1 & 1 & 0 \\ 0 & 1 & 0 & -1 & 0 & -1 \\ 0 & 0 & -1 & 0 & -1 & -1 \end{pmatrix}$$

$$e_1, e_2, e_3, e_4, e_5, e_6$$

基本回路矩阵

定义 3.4.2 当有向图 $G = \langle V, E \rangle$ 的生成树 T 确定以后，每条余树边 e 所对应的回路称为**基本回路**，该回路的方向与 e 的方向一致。由全部基本回路构成的矩阵称为 G 的**基本回路矩阵**，记为 C_f 。

例：取 $T = \{e_1, e_5, e_6\}$ ，则基本回路矩阵为：

$$C_f = \begin{pmatrix} -1 & 1 & 0 & 0 & -1 & -1 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & -1 & -1 \end{pmatrix}$$

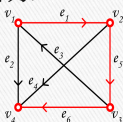
$$e_1, e_2, e_3, e_4, e_5, e_6$$

基本回路矩阵

定义 3.4.2. 当有向图 $G = \langle V, E \rangle$ 的生成树 T 确定以后，每条余树边 e 所对应的回路称为**基本回路**，该回路的方向与 e 的方向一致。由全部基本回路构成的矩阵称为 G 的**基本回路矩阵**，记为 C_f 。

例：取 $T = \{e_1, e_5, e_6\}$ ，则基本回路矩阵为：

$$C_f = \begin{matrix} \begin{bmatrix} -1 & 1 & 0 & 0 & -1 & -1 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & -1 & -1 \end{bmatrix} & \begin{matrix} T + e_2 \\ T + e_3 \\ T + e_4 \end{matrix} \\ e_1 & e_2 & e_3 & e_4 & e_5 & e_6 \end{matrix}$$



若将 C_f 的行列进行交换，使 T 的边放在后，余树边放在前，且次序与它所构成的回路一致，就可以写成**分块矩阵**形式：

$$C_f = \begin{pmatrix} 1 & 0 & 0 & -1 & -1 & -1 \\ 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & -1 & -1 \end{pmatrix}$$

$$e_2, e_3, e_4, e_1, e_5, e_6$$

即 $C_f = [I, C_{f12}]$ ，其中 C_{f12} 是树枝边所对应的子阵。

回路矩阵的性质

定理 3.4.1 有向连通图 G 的关联矩阵 B 和完全回路矩阵 C_e 的边次序一致时，恒有 $BC_e^T = 0$ 。

证明要点：设 $D = BC_e^T$ ，考察 $d_{ij} = \sum_{k=1}^m b_{ik}c_{jk}$ 。回路 C_j 与点 v_i 的关系只有 2 种：

1. C_j 不过 v_i ：与 v_i 关联的边都不是 C_j 中的边， $b_{ik} \neq 0$ 时 $c_{jk} = 0$ ，故 $d_{ij} = 0$
2. C_j 过 v_i ： C_j 必过与 v_i 关联的 2 条边 e_p, e_q 。若它们在 C_j 中方向一致，则 c_{jp}, c_{jq} 同号，但对 v_i 而言 b_{ip}, b_{iq} 反号，故 $d_{ij} = 0$ ；若在 C_j 中反向，对 v_i 同进同出，同样 $d_{ij} = 0$

定理 3.4.2 完全回路矩阵 C_e 的秩是 $m - n + 1$ 。

证明： C_f 是 C_e 的子阵且 $\text{ran } C_f = m - n + 1$ ，故 $\text{ran } C_e \geq m - n + 1$ 。由 Sylvester 定理：若 $A_{n \times s} B_{s \times m} = 0$ ，则 $\text{ran } A + \text{ran } B \leq s$ 。由 $BC_e^T = 0$ ，得 $\text{ran } B + \text{ran } C_e \leq m$ ，即 $\text{ran } C_e \leq m - n + 1$ ($\text{ran } B = n$)。故 $\text{ran } C_e = m - n + 1$ 。

定义 3.4.3 由连通图 G 中 $m - n + 1$ 个互相独立的回路组成的矩阵 C ，称为 G 的**回路矩阵**。

性质：

1. 基本回路矩阵 C_f 是回路矩阵
2. $BC^T = 0$ (边次序一致)
3. $C = PC_f$ (P 为非奇异方阵)

定理 3.4.3 回路矩阵 C 的任一 $m - n + 1$ 阶子阵行列式非 0 当且仅当这些列对应于 G 的某一棵余树。

证明：

- **充分性**：设支撑树 T ， $C_f = [I, C_{f12}]$ 。对任意回路矩阵 $C = PC_f = [P, PC_{f12}]$ ，前 $m - n + 1$ 阶子阵为 P ，非奇异，行列式非 0
- **必要性**：设 $C = [C_{11}, C_{12}]$ ，且 C_{11} 行列式非 0。若 C_{12} 对应的不是树，则 C_{12} 中有回路，即至少存在 1 行在 C_{12} 全是 0，在 C_{11} 存在非 0。此时可调整行的排列使 C_{11} 出现全 0 行， $\det C_{11} = 0$ ，矛盾

定理 3.4.4 (计算基本回路矩阵的公式) 若基本关联矩阵 B_k 和基本回路矩阵 C_f 的边次序一致，并设 $C_f = [I, C_{f12}]$ ， $B_k = [B_{11}, B_{12}]$ ，则：

$$C_{f12} = -B_{11}^T B_{12}^{-T}$$

证明： $0 = B_k C_f^T = [B_{11}, B_{12}] \begin{pmatrix} I \\ C_{f12}^T \end{pmatrix} = B_{11} + B_{12} C_{f12}^T$ ，所以 $C_{f12}^T = -B_{12}^{-1} B_{11}$ ，转置即得。

例 3.4.3：已知图 3.11 基本关联矩阵

$$B_4 = \begin{pmatrix} 1 & -1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -1 & 0 & -1 \\ -1 & 0 & 0 & 1 & 1 & 0 \end{pmatrix}$$

$e_1, e_2, e_3, e_4, e_5, e_6$

回路矩阵的性质

例3.4.3 已知图3.11基本关联矩阵

$$B_4 = \begin{bmatrix} 1 & -1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -1 & 0 & -1 \\ -1 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

e_1
 e_2
 e_3
 e_4
 e_5
 e_6

其中 e_1, e_5, e_6 所对应的矩阵行列式非零，求基本回路矩阵 C_f

其中 e_1, e_5, e_6 可构成一棵树，对 B 进行列变换：

$$B_4 = \begin{pmatrix} -1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & -1 & 0 & 0 & -1 \\ 0 & 0 & 1 & -1 & 1 & 0 \end{pmatrix} = (B_{11}, B_{12})$$

$e_2, e_3, e_4 \mid e_1, e_5, e_6$

(余树边 | 树枝边)

其中：

$$B_{11} = \begin{pmatrix} -1 & 1 & 0 \\ 1 & 0 & -1 \\ 0 & 0 & 1 \end{pmatrix}$$

，

$$B_{12} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ -1 & 1 & 0 \end{pmatrix}$$

，

$$B_{12}^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 0 & 1 \\ 0 & -1 & 0 \end{pmatrix}$$

$$C_{f12} = -B_{11}^T B_{12}^{-T} = \begin{pmatrix} 1 & 1 & 1 \\ -1 & -1 & 0 \\ 0 & -1 & -1 \end{pmatrix}$$

因此：

$$C_f = \begin{pmatrix} 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & -1 & -1 & 0 \\ 0 & 0 & 1 & 0 & -1 & -1 \end{pmatrix}$$

$$e_2, e_3, e_4, e_1, e_5, e_6$$

(8) 割集矩阵

重点：割集矩阵的定义、性质、与回路矩阵的关系、计算方法

边割集

定义 3.4.4 设 S 是图 $G = \langle V, E \rangle$ 的边子集，若：

1. $G' = \langle V, E - S \rangle$ 比 G 的连通分支数多 1
2. 对 $\forall S' \subset S$, $G'' = \langle V, E - S' \rangle$ 的连通分支数与 G 相同

则称 S 为 G 的一个（边）割集。

有向割集：对有向图，给割集 S 一个方向（ $\forall e \in S$, e 或与 S 同向，或与 S 反向）。

完全割集矩阵

定义 3.4.5 有向连通图 G 的全部割集组成的矩阵，称为**完全割集矩阵**，记为 S_e 。元素为：

$$s_{ij} = \begin{cases} 1, & e_j \in S_i \text{ 且与 } S_i \text{ 方向一致} \\ -1, & e_j \in S_i \text{ 且与 } S_i \text{ 方向相反} \\ 0, & \text{其他} \end{cases}$$

例：求右图的完全割集矩阵。

$$S_e = \begin{pmatrix} 1 & -1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & -1 & 0 & -1 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & -1 & 1 & 0 \\ 1 & -1 & 0 & 0 & -1 & -1 \\ 0 & 1 & -1 & -1 & 0 & -1 \\ 1 & 0 & 1 & -1 & 1 & 0 \end{pmatrix}$$

$$e_1, e_2, e_3, e_4, e_5, e_6$$

这些割集不一定独立，如 $S_6 = S_1 \oplus S_2$ 。

基本割集矩阵

定义 3.4.6 设 T 是连通图 G 的一棵树, e_i 是 T 的一条边, 对应 e_i 存在 G 的割集 S_i , S_i 只包含一条树上的边及某些余树边, 且与 e_i 的方向一致。称 S_i 为 G 的对应树 T 的一个**基本割集**。

定义 3.4.7 给定有向连通图 G 的一棵树, 由全部基本割集组成的矩阵称为**基本割集矩阵**, 记为 S_f 。

例: 取 $T = \{e_2, e_4, e_5\}$, 求其基本割集矩阵。

割集矩阵及其性质

例: 取 $T = \{e_2, e_4, e_5\}$, 求其基本割集矩阵。

$S_1 = \{e_1, e_2, e_3\}$ ✓
 $S_2 = \{e_1, e_4, e_6\}$ ✓
 $S_3 = \{e_3, e_5, e_6\}$ ✓
 $S_4 = \{e_2, e_4, e_5\}$
 $S_5 = \{e_1, e_2, e_5, e_6\}$
 $S_6 = \{e_2, e_3, e_4, e_6\}$
 $S_7 = \{e_1, e_3, e_4, e_5\}$

$S_f = \begin{bmatrix} S_1 \\ S_2 \\ S_3 \end{bmatrix} = \begin{bmatrix} -1 & 1 & -1 & 0 & 0 & 0 \\ -1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$

$e_1 \quad e_2 \quad e_3 \quad e_4 \quad e_5 \quad e_6$

$$S_f = \begin{pmatrix} -1 & 1 & -1 & 0 & 0 & 0 \\ -1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 \end{pmatrix}$$

$e_1, e_2, e_3, e_4, e_5, e_6$

若将余树边对应的列放在前, 树枝边对应的列放在后, 且与割集次序一致:

$$S_f = \begin{pmatrix} -1 & -1 & 0 & 1 & 0 & 0 \\ -1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{pmatrix}$$

$e_1, e_3, e_6, e_2, e_4, e_5$

$$= [S_{f11}, I]$$

其中 I (单位矩阵) 对应一棵树, 显然 $\text{ran } S_f = n - 1$ 。

割集矩阵的性质

定理 3.4.5 当有向连通图 G 的完全回路矩阵 C_e 和完全割集矩阵 S_e 的边次序一致时, 有 $S_e C_e^T = 0$ 。

证明要点: 割集 S_i 与回路 C_j 的关系只有 2 种:

1. S_i 与 C_j 不相交: C_j 中的边在 S_i 中不出现, $d_{ij} = 0$
2. S_i 与 C_j 相交: 有偶数条共同的边, 相邻两条边 e_p, e_q 为一对。若在 S_i 中方向一致, 则在 C_j 中方向相反, 反之亦然, 故 $d_{ij} = 0$

定理 3.4.6 完全割集矩阵 S_e 的秩是 $n - 1$ 。

证明: 显然 $\text{ran } S_e \geq n - 1$ 。由 $S_e C_e^T = 0$ 及 Sylvester 定理, $\text{ran } S_e + \text{ran } C_e \leq m$ 。而 $\text{ran } C_e = m - n + 1$, 故 $\text{ran } S_e \leq n - 1$ 。所以 $\text{ran } S_e = n - 1$ 。

定义 3.4.8 连通图 G 的 $n - 1$ 个互相独立的割集构成的矩阵称为 G 的**割集矩阵**, 记为 S 。

性质:

1. 基本割集矩阵 S_f 是割集矩阵
2. $SC^T = 0$ (S 和 C 的边次序一致)
3. $S = PS_f$ (P 为非奇异方阵)

定理 3.4.7 割集矩阵 S 的任一 $n - 1$ 阶子阵行列式非 0 当且仅当这些列对应于 G 的某棵树。

定理 3.4.8 (割集矩阵与回路矩阵的关系) 若基本割集矩阵 $S_f = [S_{f11}, I]$ 和基本回路矩阵 $C_f = [I, C_{f12}]$ 的边次序一致, 则:

$$S_{f11} = -C_{f12}^T$$

即基本割集矩阵中的余树边部分 = 基本回路矩阵中树枝边部分的**负转置**。

推论 3.4.1 基本割集矩阵与基本关联矩阵的关系：

$$S_{f11} = B_{11}^{-1} B_{12}$$

(可用来求基本割集矩阵 S_f)

矩阵关系总结 (必考!)

三个矩阵的核心关系：

$$BC^T = 0$$

$$SC^T = 0$$

已知基本关联矩阵 B_k ，求基本回路矩阵 C_f ：

$$C_f = [I, C_{f12}]$$

，其中 $C_{f12} = -B_{11}^T B_{12}^{-T}$

已知基本回路矩阵 C_f ，求基本割集矩阵 S_f ： $S_f = [S_{f11}, I]$ ，其中 $S_{f11} = -C_{f12}^T$

已知基本关联矩阵 B_k ，求基本割集矩阵 S_f ： $S_f = [S_{f11}, I]$ ，其中 $S_{f11} = B_{11}^{-1} B_{12}$

计算步骤：

1. 确定生成树 T ，将关联矩阵的列重排为余树边 | 树枝边
2. 分块得到 $B_k = [B_{11}, B_{12}]$
3. 求 B_{12}^{-1} ，计算 $C_{f12} = -B_{11}^T B_{12}^{-T}$
4. 得到 $C_f = [I, C_{f12}]$
5. 得到 $S_f = [-C_{f12}^T, I]$

(9) 最短树

最小支撑树

定义：一棵支撑树中的所有边的权之和称为该支撑树的**权**。具有最小权的支撑树，称为**最小支撑树** (Minimal Spanning Tree)。

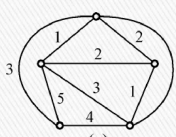
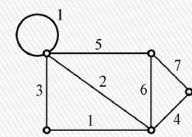
Prim 算法

基本思想：首先任选一个节点 v_0 构成集合 V' ，然后不断在 $V - V'$ 中选一条到 V' 中某点最短的边 (u, v) 进入树 T ，并令 $V' = V' \cup \{u\}$ ，直至 $V' = V$ 。与 Dijkstra 算法类似。

例：求下图最小生成树。

填空题 2分
⚙️ 设置

用Kruskal算法生成下图最小生成树，
图(a)的最小支撑树的权值和为 [填空1]，
图(b)的最小支撑树的权值和为 [填空2]。

正常使用填空题需3.0以上版本雨课堂
作答

步骤:

1. 初始化: $\Pi(a) = \infty, \Pi(b) = 5, \Pi(c) = 8, \Pi(e) = 2, \Pi(z) = 6$
2. 选 e ($\Pi = 2$): 更新 $\Pi(c) = 8 \rightarrow 4, \Pi(z) = 6 \rightarrow 3$
3. 选 z ($\Pi = 3$)
4. 选 c ($\Pi = 4$): 更新 $\Pi(a) = \infty \rightarrow 2, \Pi(b) = 5 \rightarrow 1$
5. 选 b ($\Pi = 1$)
6. 选 a ($\Pi = 2$)
7. 最小生成树权值 $= 2 + 3 + 4 + 1 + 2 = 12$? 不对, 应该是 $1 + 2 + 2 + 3 + 4 = 12$

定理 3.6.3 设 V' 是赋权连通图 $G = (V, E)$ 的节点真子集, e 是二端点分跨在 V' 和 $V - V'$ 的最短边, 则 G 中一定存在包含 e 的最短树 T 。

证明: 设 T_0 是一棵最短树。若 $e \notin T_0$, 则 $T_0 + e$ 构成唯一回路, 该回路一定包含 e 和 $e' = (u, v)$ (其中 $u \in V', v \in V - V'$)。由条件 $w(e) \leq w(e')$, 作 $T_0 \oplus (e, e')$, 得到的仍然是最短树。因此 Prim 算法得到的是最小树。

Kruskal 算法

基本思想: 不断往 T 中添加当前的最短边, 如果此时会构成回路, 那么它一定是这个回路的最长边, 删之。直至最后达到 $n - 1$ 条边为止。

步骤:

1. $T = \emptyset$, E 将 m 条边按权从小到大排序为 $e_1, e_2, \dots, e_m$
2. 对 $i = 1$ 到 $n - 1$: 在 E 中按顺序取边, 直到找到不与 T 中的边构成回路的边 e_j , $T = T \cup \{e_j\}$

关键实现: **并查集**。首先将 G 的 n 个顶点看成 n 个孤立的连通分支, 依边权递增顺序查看每条边 (v, w) : 若 v 和 w 分别是当前 2 个不同连通分支中的顶点, 就用边 (v, w) 将它们连接; 若在同一个连通分支中, 跳过。

两种方法均为**贪心算法**, 求出的都是最优解:

1. Kruskal 算法复杂度 $O(m + p \log m)$ (p 为迭代次数, 介于 n 到 m 之间)
2. Prim 算法复杂度 $O(n^2)$
3. 稠密图用 Prim, 稀疏图用 Kruskal

(10) Huffman 树

重点: 二叉树定义、WPL 计算、Huffman 算法构造最优二叉树、前缀码

二叉树

定义 3.5.1 除树叶外, 其余节点的正度最多为 2 的外向树称为**二叉树**。除树叶节点外, 其余节点的正度都是 2, 称为**完全二叉树**。

最优二叉树

定义 3.5.2 给一棵完全二叉树的所有树叶都赋予权重 w_i , 称之为**赋权二叉树**, 且将从根节点到树叶节点 v_i 的路径长度记作 l_i , 则该树的**带权路径长度**:

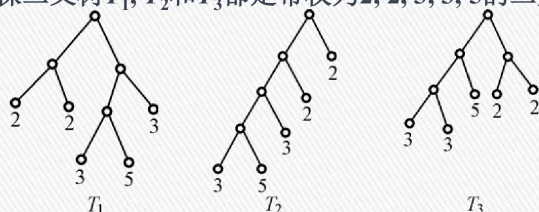
$$WPL = \sum_{i \in \text{leaves}} w_i l_i$$

定义 3.5.3 在树叶节点个数以及它们对应权值相同的情况下, 构造出的所有赋权二叉树中**带权路径长度最短**的树称为**最优二叉树**。

例: 三棵带权为 2, 2, 3, 3, 5 的二叉树 T_1, T_2, T_3 :

最优二叉树

下面三棵二叉树 T_1 , T_2 和 T_3 都是带权为 2, 2, 3, 3, 5 的二叉树



根据树权 $W(T)$ 的定义, 计算出它们的权分别为:

$$W(T_1) = 2 \times 2 + 2 \times 2 + 3 \times 3 + 5 \times 3 + 3 \times 2 = 38$$

$$W(T_2) = 3 \times 4 + 5 \times 4 + 3 \times 3 + 2 \times 2 + 2 \times 1 = 47$$

$$W(T_3) = 3 \times 3 + 3 \times 3 + 5 \times 2 + 2 \times 2 + 2 \times 2 = 36$$

最优二叉树唯一吗? 如何构造?

$$W(T_1) = 2 \times 2 + 2 \times 2 + 3 \times 3 + 5 \times 3 + 3 \times 2 = 38$$

$$W(T_2) = 3 \times 4 + 5 \times 4 + 3 \times 3 + 2 \times 2 + 2 \times 1 = 47$$

$$W(T_3) = 3 \times 3 + 3 \times 3 + 5 \times 2 + 2 \times 2 + 2 \times 2 = 36$$

注意: 计算 WPL 时, l_i 是根到叶节点的路径长度 (边数), 不是节点数!

Huffman 算法

算法步骤:

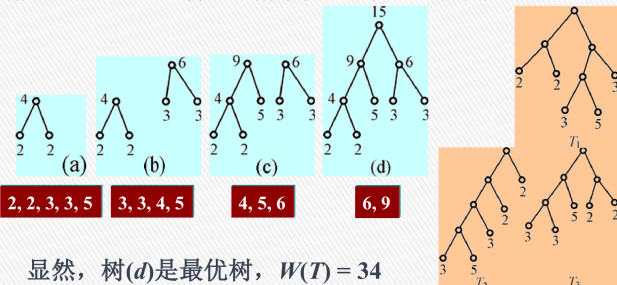
1. 根据所有权重 $\{w_1, w_2, \dots, w_n\}$ 构造二叉树集合 $\{T_1, T_2, \dots, T_n\}$, 每棵 T_i 仅有一个权重为 w_i 的节点
2. 将这些 w_i 按从小到大排序, 放入优先级队列 Q 中
3. 从 Q 中连续弹出两次队首元素, 将对应的二叉树分别作为左右子树合并, 合并后根节点权重 = 左子树根权重 + 右子树根权重, 将新树插入 Q 中
4. 重复步骤 3, 直到 Q 为空
5. 整个算法复杂度 $O(n \log n)$

例: 求带权 2, 2, 3, 3, 5 的最优二叉树。

Huffman 树

例: 求带权 2, 2, 3, 3, 5 的最优二叉树

解: 利用 Huffman 算法求解最优二叉树的全过程如下:



显然, 树(d)是最优树, $W(T) = 34$
右边三棵树都不是最优二叉树。

合并过程: 2, 2, 3, 3, 5 $\rightarrow$ 3, 3, 4, 5 $\rightarrow$ 4, 5, 6 $\rightarrow$ 6, 9

$$W(T) = 2 \times 3 + 2 \times 3 + 3 \times 2 + 3 \times 2 + 5 \times 2 = 34$$

定理 3.5.1 由 Huffman 算法得到的二叉树是最优二叉树。

证明要点:

1. 对于最优二叉树 T , $w_1 \leq w_2 \leq \dots \leq w_n$, 则 $l_1 = \max l_i$ (否则可以交换得更优)
2. w_1 一定有兄弟节点, 且该兄弟一定是 w_2
3. w_1 和 w_2 及其父节点构成最优子树, 合并为一个节点 (权重相加) 不改变最优性
4. 重复合并直到三个节点

Huffman 编码

前缀码: 设 $A = \{b_1, b_2, \dots, b_m\}$ 为符号串集合, 若 $\forall b_i, b_j \in A, i \neq j$, b_i 和 b_j 互不为前缀, 则称 A 为**前缀码**。
若符号串中只出现 0 和 1, 则称为**二元前缀码**。

可用二叉树产生二元前缀码: 设 T 是具有 k 个叶节点的二叉树, v 为内点, 若 v 有两个孩子, 左边标 0, 右边标 1。从根到叶节点的通路上各边的标号组成的符号串即为该叶节点的编码。因为根到树叶的路径唯一且不是另一条路径的一部分, 所得必为前缀码。

例: 在通信中, 八进制数字出现的频率为 0:25%, 1:20%, 2:15%, 3:10%, 4:10%, 5:10%, 6:5%, 7:5%。求最佳前缀码。

权序列 (乘 100): $w_7 = 5, w_6 = 5, w_5 = 10, w_4 = 10, w_3 = 10, w_2 = 15, w_1 = 20, w_0 = 25$

Huffman树的应用

例: 在通信中, 八进制数字出现的频率如下:

0: 25%	1: 20%	2: 15%	3: 10%
4: 10%	5: 10%	6: 5%	7: 5%

解 (续)

用 Huffman 算法求最优树如右图(a)。

图中矩形框中的符号串为对应数字的编码, 即可用编码 01, 11, 001, 100, 101, 0001, 00000 和 00001 来分别传输 0, 1, 2, 3, 4, 5, 6 和 7。

(a)

8 个编码的集合 {01, 11, 001, 100, 101, 0001, 00000, 00001} 为前缀码, 且是最佳前缀码。

编码: {01, 11, 001, 100, 101, 0001, 00000, 00001} 分别对应 0, 1, 2, 3, 4, 5, 6, 7

$W(T) = 285$ (传输 100 个八进制数字需要 285 个二进制位)

传输 10^n 个数字: Huffman 编码需要 2.85×10^n 位, 等长编码 (3 位) 需要 3×10^n 位。

最佳前缀码不唯一 (选择最小权和左右位置不唯一), 但 WPL 都相等。

六、平面图与对偶图 2026-04-23

(1) 平面图

平面图定义 (定义 4.1.1)

1. **可嵌入平面 / 可平面图:** 若能把图 G 画在一个平面上, 使任何两条边都不相交, 就称 G 可嵌入平面
2. **平面图:** 可平面图在平面上的一个嵌入
3. **非平面图:** 没有平面嵌入的图

Jordan 曲线: 一条连续的、自身不相交的、起点和终点相重合的曲线。

Jordan 曲线定理: 连接 “int” J 的点和 “ext” J 的点的任何连线必在某点和 J 相交。

面 (定义 4.1.2)

设 G 是一个平面图, 由它的若干边所构成的一个区域, 若区域内不含任何节点及边, 就称该区域为 G 的一个**面 (域)**。

1. 面积无限的面称为**无限面 (外部面)**
2. 面积有限的面称为**有限面 (内部面)**
3. 包围这个域的诸边称为该面的**边界**
4. 边界的长度称为该面的**次数**, 记为 $\deg(R)$
5. 割边在面的次数中被计算 2 次

例：右图有 4 个面。 $\deg(R_1) = 1, \deg(R_2) = 3, \deg(R_3) = 2, \deg(R_0) = 8$

面的次数定理

定理 4.1.3 平面嵌入图 G 中所有面的次数之和等于边数 m 的两倍，即 $\sum_{i=1}^r \deg(R_i) = 2m$ 。

证明：每条边要么是两个面的公共边界（各计算 1 次），要么是割边只在一个面的边界上出现（计算 2 次），所以每条边总被计算 2 次。

欧拉公式（定理 4.1.1）：对于任意连通平面图 G ，有 $n - m + r = 2$ 。其中 n, m, r 分别为 G 的顶点数、边数和面数。

推论 4.1.1（推广）：对于具有 k 个连通分支的平面图 G ，有 $n - m + r = k + 1$ 。

推论 4.1.2：对一般平面图 G ，恒有 $n - m + r \geq 2$ 。

定理 4.1.2：设简单连通平面图没有割边，且每个域的边界数至少为 t ，则 $m \leq \frac{t(n-2)}{t-2}$ 。

证明： $2m = \sum \deg(R_i) \geq t \cdot r$ ，由 $r = 2 + m - n$ 代入得 $2m \geq t(2 + m - n)$ ，整理即得。

用此定理可证明 K_5 和 $K_{3,3}$ 都不是平面图：

1. K_5 : $m = 10 > \frac{3(5-2)}{3-2} = 9$ ，矛盾
2. $K_{3,3}$: 最短圈长度 ≥ 4 , $m = 9 > \frac{4(6-2)}{4-2} = 8$ ，矛盾

(2) 极大平面图

定义与性质

定义 4.2.1 设 G 为 $n \geq 3$ 的简单平面图，若在 G 的任意不相邻的节点 u, v 之间加边 (u, v) ，所得图为非平面图，则称 G 为**极大平面图**。

性质：

1. 若简单平面图中已无不相邻顶点，则是极大平面图（如 K_1, K_2, K_3, K_4 ）
2. 极大平面图是连通的
3. **性质 3：**简单连通平面图 $G(n \geq 3)$ 为极大平面图的充要条件是每个面的次数均为 3
4. **性质 4：** $3r = 2m$ ，其中 m 为边数， r 为面数

定理 4.2.1 设 G 是 $n(n \geq 3)$ 阶 m 条边的极大平面图，则 $m = 3n - 6, r = 2n - 4$ 。

证明：由欧拉公式 $r = 2 + m - n$ ，由极大平面图性质 $2m = 3r$ ，代入整理即得。

推论 4.2.1：设 G 是 $n(n \geq 3)$ 阶 m 条边的简单连通平面图，则 $m \leq 3n - 6, r \leq 2n - 4$ 。

推论：简单平面图不含 K_3 子图时 $m \leq 2n - 4$ （每个域边界数至少为 4）。

定理 4.2.2：简单连通平面图的最小度 $\delta \leq 5$ 。

(3) 图的平面性检测

非平面图

K_5 是节点数最少的非平面图， $K_{3,3}$ 是边数最少的非平面图。

定理 4.3.1 若图 G 是平面图，则 G 的任何子图都是平面图。**定理 4.3.2** 若 G 是非平面图，则 G 的任何母图也都是非平面图。**推论 4.3.1** $K_n(n \geq 5)$ 和 $K_{3,n}(n \geq 3)$ 都是非平面图。**定理 4.3.3** 重边和自环不影响图的平面性。

同胚与边收缩

定义 4.3.1（同胚）：若两个图 G_1 与 G_2 同构，或通过反复插入或消去 2 度顶点后是同构的，则称 G_1 与 G_2 是**同胚**的。

定义 4.3.3（边收缩）：设 $e = (u, v)$ 为图 G 的一条边，在 G 中删除 e ，增加新的顶点 w ，使 w 关联除 e 外的 u, v 关联的一切边。

Kuratowski 定理

库拉图斯基定理 1：图 G 是平面图，当且仅当 G 中不含与 K_5 和 $K_{3,3}$ 同胚的子图。

库拉图斯基定理 2：图 G 是平面图，当且仅当 G 中没有可收缩到 K_5 和 $K_{3,3}$ 的子图。

例 4.3.2：彼得松图不是平面图。将边 $(a, f), (b, g), (c, h), (d, i), (e, j)$ 收缩后得到 K_5 。

极小非平面图：若 G 是非平面图，并且任意删除一条边所得图为平面图。 $K_5, K_{3,3}$ 都是极小非平面图。

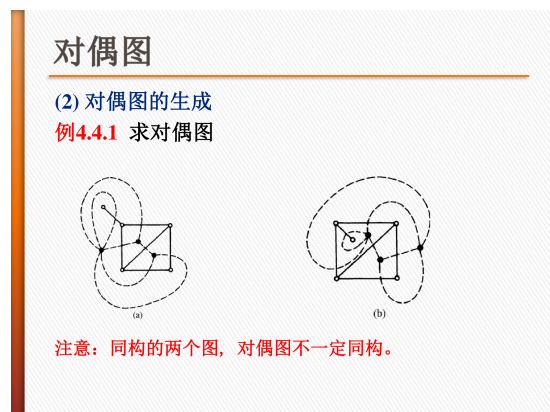
(4) 对偶图

对偶图的定义

定义 4.4.1 设 G 是平面图的某个平面嵌入，构造 G 的对偶图 G^* 如下：

1. 在 G 的面 f_i 中放置 G^* 的顶点 v_i^*
2. 若 e 在 G 的面 f_i 与 f_j 的公共边界上，做 G^* 的边 $e^*(v_i^*, v_j^*)$ 与 e 相交，且 e^* 不与其他边相交
3. 若 e 在面 f_i 之内，则 $e^*(v_i^*, v_i^*)$ 是以 v_i^* 为端点的环

注意：只有平面图才有对偶图。同构的两个图，对偶图不一定同构。



对偶图的性质

1. **性质 4.4.1**：如果 G 是平面图， G 一定有对偶图 G^* ，而且 G^* 是唯一的
2. **性质 4.4.2**： G^* 是连通图
3. **性质 4.4.3**： $m = m^*, n = r^*, r = n^*$ (G 与 G^* 的节点、边和域之间的对应)
4. **性质 4.4.4**： $d_{G^*}(v_i^*) = \deg(R_i)$ (对偶图中顶点的度数 = 原图中面的次数)
5. **性质 4.4.5**：若 C 是 G 的初级回路，则 S^* 是 G^* 中的割集

定理 4.4.1： G 有对偶图的充要条件是 G 为平面图。

证明必要性（非平面图没有对偶图）：由 Kuratowski 定理，只需证 K_5 和 $K_{3,3}$ 没有对偶图。

1. K_5 ： $m = 10, n = 5, r \geq 7$ ，若有对偶图 $m^* = 10, n^* \geq 7$ 。由于 K_5 型子图无重边自环， $d(v_i^*) \geq 3$ ， $\sum d(v_i^*) \geq 3 \times 7 = 21 > 2m^* = 20$ ，矛盾
2. $K_{3,3}$ ： $m = 9, n = 6, r \geq 5$ ，每个面边界至少 4， $d(v_i^*) \geq 4$ ， $\sum d(v_i^*) \geq 4 \times 5 = 20 > 2m^* = 18$ ，矛盾

自对偶图

定义 4.4.2 设 G^* 是平面图 G 的对偶图，若 $G^* \cong G$ ，则称 G 为**自对偶图**。

轮图 (n 阶)：在 $n-1$ ($n \geq 4$) 边圈 C_{n-1} 内放置一个顶点，使该顶点与 C_{n-1} 上的所有顶点均相邻。轮图都是自对偶图。